200417ilt

CLOUDERA

Educational Services

Data Analyst for Cloudera Data Center: Hands-On Exercises

Table of Contents

General Notes 1
Hands-On Exercise: Launching the Exercise Environment 3
Hands-On Exercise: Data Ingest with Hadoop Tools 7
Hands-On Exercise: Running Queries from Shells, Scripts, and Hue 15
Hands-On Exercise: Using Functions 21
Hands-On Exercise: Data Management 25
Hands-On Exercise: Data Storage and Performance 33
Hands-On Exercise: Working with Multiple Datasets 36
Hands-On Exercise: Analytic Functions 38
Hands-On Exercise: Complex Data 40
Hands-On Exercise: Analyzing Text with Hive 44
Hands-On Exercise: Hive Optimization 49
Hands-On Exercise: Impala Optimization 53
Hands-On Exercise: Data Transformation with Hive 57
Optional Hands-On Exercise: Analyzing Abandoned Carts 63
Hands-On Exercise: Using Apache Kudu with Apache Impala 69
Appendix: Data Model Reference 76

Appendix: Regular Expression Reference 80
Appendix: Troubleshooting Tips 83
Appendix: Continuing Exercises After Class 86

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

General Notes

Cloudera provides a *hands-on environment* to accompany this training course. This consists of a virtual machine (VM) running Linux, with a recent version of CDP already installed and configured for you. This VM runs in pseudo-distributed mode, a configuration that enables a Hadoop cluster to run on a single machine.

Restarting Required Services

Your hands-on environment is configured to start all required services when you start the cluster. (See "Hands-On Exercise: Launching the Exercise Environment" in this manual.) Under certain circumstances, services may fail and need to be restarted. If you encounter messages such as `Connection refused`, `Not connected`, or `Could not connect`, or if jobs or queries hang indefinitely and never complete, refer to the "Appendix: Troubleshooting Tips" section of this manual.

Points to Note while Working in the Hands-On Environment

1. The hands-on environment is set to log in as the user `training`. If you log out, you can log back in as the user `training` with the password `training`. The root password is also `training`, though you can prefix any command with `sudo` to run it as root.

---

2. Exercises often contain steps with commands that look like this:

```
$ hdfs dfs -put accounting_reports_taxyear_2016 \
 /user/training/tax_analysis/
```

The dollar sign (`$`) represents the command prompt. Do *not* include this character when copying and pasting commands into your terminal window. Also, the backslash (`\`) signifies that the command continues on the next line. You may either enter the code as shown (on two lines), or omit the backslash and type the command on a single line.

> **HDFS Warnings**
>
> Due to a bug in HDFS, you may see `java.lang.InterruptedException` warnings when running `hdfs dfs -put` commands or other commands that copy

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

data to HDFS. These warnings are of no consequence and can be ignored.

3. Although many students are comfortable using UNIX text editors like vi or emacs, some might prefer a graphical text editor. To invoke the graphical editor from the command line, type `pluma` followed by the path of the file you wish to edit. Appending & to the command allows you to type additional commands while the editor is still open. Here is an example of how to edit a file named `myfile.txt`:

```
$ pluma myfile.txt &
```

Points to Note during the Exercises

Sample Solutions

If you need a hint or want to check your work, the `solution` subdirectory within each exercise directory contains complete code samples.

Catch-Up Script

If you are unable to complete an exercise, we have provided a script to catch you up automatically. Each exercise has instructions for running the catch-up script.

`$ADIR` Environment Variable

`$ADIR` is a shortcut that points to the `/home/training/training_materials/analyst` directory, which contains the code and data you will use in the exercises.

Fewer Step-by-Step Instructions as You Work through These Exercises

As the exercises progress, and you gain more familiarity with the tools you're using, we provide fewer step-by-step instructions. You should feel free to ask your instructor for assistance at any time, or to consult with your fellow students.

Bonus Exercises

Many of the exercises contain one or more optional "bonus" sections. We encourage you to work through these if time remains after you finish the main exercise and you would like an additional challenge to practice what you have learned.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

Hands-On Exercise: Launching the Exercise Environment

In this exercise, you will start your exercise environment and the cluster on which you will do the course exercises.

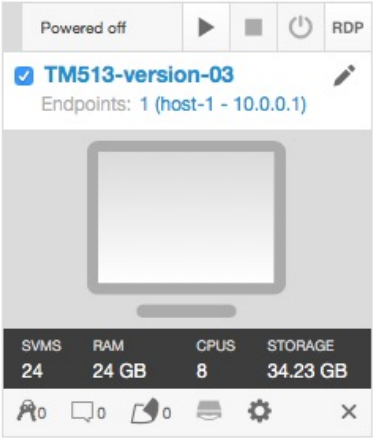
This course provides a single-host exercise environment running a pseudo-distributed Cloudera cluster (that is, a cluster running on a single host that emulates a multi-host environment) to complete the exercises. This section will walk you through starting the host in your environment, then starting the cluster within the environment.

Start Your Exercise Environment

1. In your local browser, open the URL provided by your instructor to view the exercise environment portal.

---

2. The environment portal page displays a thumbnail image for your exercise environment remote host. The host may be powered off initially, indicated by a gray background; or running, indicated by a green background. (The machine name version may be different in your environment than the one below.)



   Click the host icon to open a new window showing the remote host machine. If the machine is not running, start it by clicking the play button (triangle icon). It will take a few minutes to start.

---

3. When it is fully started, the remote host's desktop will display. These exercises refer to this as the "remote desktop" to distinguish it from your own local machine's desktop.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

The remainder of the exercises in the course will be performed on the remote desktop.

Verify Your Cluster Services

After your machine has been running for a few minutes, confirm that your exercise environment is set up and the cluster services are running correctly by reviewing the cluster status using Cloudera Manager.

4. Start the Firefox browser using the icon on the remote desktop, then click the **Cloudera Manager** bookmark.

Note: If you get an error message, Cloudera Manager probably has not finished starting. Wait a minute and try again.

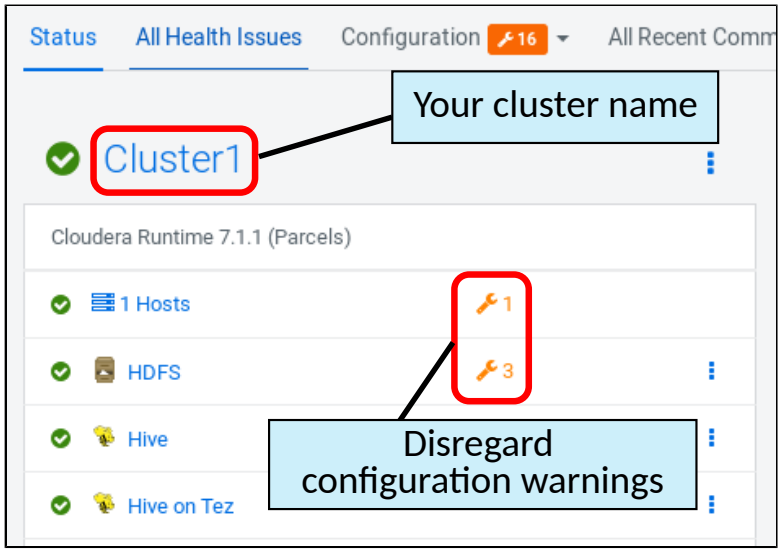
---

5. Log in as user **admin** with password **admin**.

---

6. Verify that all the services in your cluster are healthy (indicated by a green dot), as shown below. You may disregard yellow configuration warning icons.

Note: *Some health warnings may appear as the cluster is starting. They will typically resolve themselves within five minutes after the Start Cluster script finishes. Please be patient.* If health issues remain, refer to the tip entitled "Cloudera Manager displays unhealthy status of services" in Appendix: Troubleshooting Tips.



Your cluster may not be running exactly the services shown. This is okay and will not interfere with completing the exercises.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

Your exercise environment is now ready.

See Stopping Your Exercise Environment below for instructions on stopping and restarting your environment.

Optional: Download and View the Exercise Manual on Your Remote Desktop

In order to be able to copy and paste from the Exercise Manual (the document you are currently viewing) to your remote desktop, you might want to view the document on your remote machine rather than on your local machine.

7. Download the Exercise Manual

a. On your remote desktop, start the Firefox browser using the shortcut icon in the menu bar.

b. Log in to your Cloudera University account by going to `https://university.cloudera.com/user/learning/enrollments` and clicking **Returning Student Login**. Then from the Dashboard, find this course under **Current**.

c. Select the course title, then click to download the Exercise Manual under **Materials**. This will save the Exercise Manual PDF file in the `Downloads` folder in the training user's home directory.

---

8. View the Exercise Manual on Your Remote Host

a. Open a terminal window using the shortcut on the remote desktop, and then start the Atril document viewer:

```
$ atril &
```

b. In Atril, select menu item **File** > **Open** and open the Exercise Manual PDF file in the `Downloads` directory.

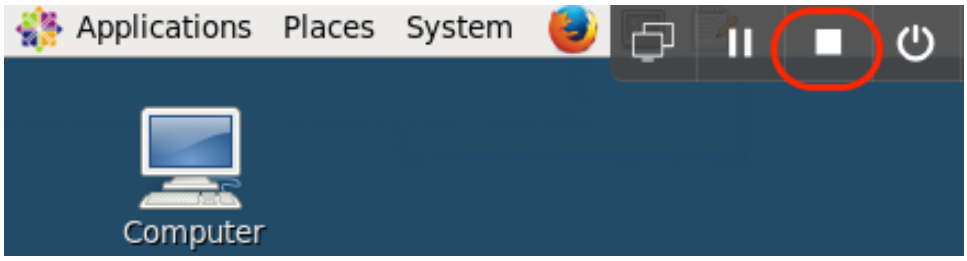
Stopping Your Exercise Environment

Your exercise environment will be suspended at the end of class, but it will continue to be available for 10 hours of use or 30 days, whichever comes first. In order to use the environment, you will need to restart the remote host following the instructions in the Start Your Exercise Environment section.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

In order to minimize your usage of the limited time available, your remote host will be automatically suspended after thirty minutes of idle time. You can also stop it manually.

In the toolbar at the top of the remote desktop display, click the stop button (a square).



This is the end of the exercise.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

Hands-On Exercise: Data Ingest with Hadoop Tools

Files and Data Used in This Exercise	
Exercise directory	`$ADIR/exercises/data_ingest`
Files	`$ADIR/data/access.log`
MySQL tables	`employees, customers, products, orders, order_details`
Hive/Impala tables	`customers, products, orders, order_details`

In this exercise, you will practice using the Hadoop command line utility to interact with Hadoop's Distributed Filesystem (HDFS) and use Sqoop to import tables from a relational database to HDFS.

Exploring HDFS using the Hue File Browser

1. Launch the hands-on environment if you haven't already done so, and start the Firefox web browser in the hands-on environment by clicking on the icon in the system toolbar:

---

2. In Firefox, click the **Hue** bookmark in the bookmark toolbar (or type **http://localhost.localdomain:8889/hue/** into the address bar and press the `Enter` key.)

---

3. After a few seconds, you should see Hue's home screen. Enter **training** in both the username and password fields, and then click the **Sign In** button.

   Note: You can close the tour that appears. These exercises will present the features you need for this course.

---

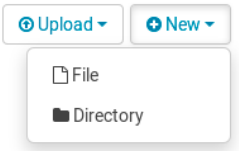
4. Click the Files icon in the sidebar on the left. (The icon appears as a pair of documents.) The File Browser displays your HDFS home directory (since your user ID on the cluster is `training`, your home directory in HDFS is `/user/training`). There is no user data in your home directory yet.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

> **In Case of Error**
>
> If Hue displays the error message Cannot access: /user/training/, this indicates that one or more of the required services has failed and needs to be restarted. Refer to the above section "Restarting Required Services" for instructions about how to restart the required services.

---

5. Create a temporary sub-directory: select the **+New** menu and click **Directory**.



---

6. Enter directory name **test** and click the **Create** button. Your home directory now contains a directory called `test`.

---

7. Click **test** to view the contents of that directory; currently it contains no files or subdirectories.

---

8. Upload a file to the directory. Start by selecting **Upload**.

---

9. Click **Select files** to bring up a file browser. By default, the `/home/training/Desktop` folder displays. Click the home directory button (**training**) then navigate to the course data directory: `training_materials/analyst/data`.

---

10. Choose any of the data files in that directory, *except* `access.log`. (Hue limits file uploads to 64MB, so you must upload larger files using the command line.) Then click the **Open** button.

> **In Case of Error**
>
> If Hue displays the error message Error: IOException: Failed to find datanode, this indicates that one or more of the required services has failed and needs to be restarted. Refer to the above section "Restarting Required Services" for instructions about how to restart the required services.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

11. The file you selected will be loaded into the current HDFS directory. Click the filename to see the file's contents. Because HDFS is designed to store very large files, Hue will not display the entire file, just the first page of data. You can use the scrollbar to see more of the data.

---

12. Return to the `test` directory by clicking **Back** on the left side of the main panel.

---

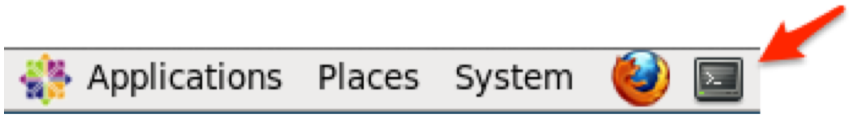
13. Above the list of files in your current directory is the full path of the directory you are currently displaying. You can click on any directory in the path, or on the first slash (/) to go to the top level (root) directory. Click **training** to return to your home directory.

---

14. Delete the temporary `test` directory you created, including the file in it, by selecting the checkbox next to the directory name then clicking the **Move to trash** button. Confirm that you want to delete by clicking **Yes**. Your home directory now shows a `.Trash` subdirectory.

Exploring HDFS Using the Command Line

15. You can use the `hdfs dfs` command to interact with HDFS from the command line. Close or minimize Firefox, and then open a terminal window by clicking the icon in the system toolbar:



---

16. In the terminal window, enter:

```
$ hdfs dfs
```

This displays a help message describing all subcommands associated with `hdfs dfs`.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

17. Run the following command:

```
$ hdfs dfs -ls
```

The `-ls` flag says to list the contents of a directory. Since no path is specified, `hdfs dfs` assumes you are referring to your home directory. This output will list the current contents of your home directory (`.Trash` and possibly a staging subdirectory).

---

18. Now run the following command:

```
$ hdfs dfs -ls /
```

This lists the contents of the HDFS root directory, specified by the path `/`. One of the directories listed is `/user`. Each user on the cluster has a "home" directory below `/user` corresponding to his or her user ID.

---

19. Note the `/analyst` directory. Most of your work in this course will be in that directory. Try creating a temporary subdirectory in `/analyst`:

```
$ hdfs dfs -mkdir /analyst/test1
```

---

20. Next, add a web server log file to this new directory in HDFS:

```
$ hdfs dfs -put $ADIR/data/access.log /analyst/test1/
```

Overwriting Files in Hadoop

Unlike the UNIX shell, Hadoop won't overwrite files and directories. This feature helps protect users from accidentally replacing data that may have taken hours to produce. If you need to replace a file or directory in HDFS, you must first remove the existing one. Please keep this in mind in case you make a mistake and need to repeat a step during the Hands-On Exercises.

To remove a file:

```
$ hdfs dfs -rm /analyst/example.txt
```


© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

To remove a directory and all its files and subdirectories (recursively):

```
$ hdfs dfs -rm -r /analyst/example/
```

---

21. Verify the last step by listing the contents of the `/analyst/test1` directory. You should observe that the `access.log` file is present and occupies 106,339,468 bytes of space in HDFS:

```
$ hdfs dfs -ls /analyst/test1
```

---

22. Remove the temporary directory and its contents:

```
$ hdfs dfs -rm -r /analyst/test1
```

Importing Database Tables into HDFS with Sqoop

Dualcore stores information about its employees, customers, products, and orders in a MySQL database. In the next few steps, you will examine this database before using Sqoop to import its tables into HDFS.

23. In a terminal window, log in to MySQL and select the `analyst_dualcore` database:

```
$ mysql --user=training --password=training analyst_dualcore
```

Note: The exercise environment uses MariaDB as its MySQL database. If you are used to MySQL, the only difference you should notice is the prompt, which says `MariaDB` instead of `mysql`.

---

24. Next, list the available tables in the `analyst_dualcore` database (`MariaDB>` represents the MySQL client prompt and is not part of the command):

```
MariaDB> SHOW TABLES;
```

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

25. Review the structure of the `employees` table and examine a few of its records:

```
MariaDB> DESCRIBE employees;
MariaDB> SELECT * FROM employees LIMIT 10;
```

---

26. Exit MySQL by typing **quit**, and press the `Enter` key:

```
MariaDB> quit
```

> **Data Model Reference**
>
> For your convenience, you will find a reference section depicting the structure for the tables you will use in the exercises at the end of this Exercise Manual.

---

27. Run the following command, which imports the `employees` table into the `/analyst/dualcore` directory in HDFS, using tab characters to separate each field:

```
$ sqoop import \
 -Dorg.apache.sqoop.splitter.allow_text_splitter=true \
 --connect jdbc:mysql://localhost/analyst_dualcore \
 --username training --password training \
 --fields-terminated-by '\t' \
 --warehouse-dir /analyst/dualcore \
 --table employees
```

It is normal for Sqoop to produce a lot of log messages, which are shown in your terminal window, during the import process.

> **Hiding Passwords**
>
> Typing the database password on the command line is a potential security risk since others may see it. An alternative to using the `--password` argument is to use `-P` and let Sqoop prompt you for the password, which is then not visible when you type it.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

> **Sqoop Code Generation**
>
> After running the `sqoop import` command above, you may notice a new file named `employees.java` in your local directory. This is an artifact of Sqoop's code generation and is really only of interest to Java developers, so you can ignore it.

---

28. Revise the previous command and import the `customers` table into HDFS. Note that the `-Dorg.…` is not needed (the primary key is not a character string), but you can include it if you like.

---

29. Revise the previous command and import the `products` table into HDFS.

---

30. Revise the previous command and import the `orders` table into HDFS.

---

31. Next, import the `order_details` table into HDFS. The command is slightly different because this table only holds references to records in the `orders` and `products` table, and it lacks a primary key of its own. Consequently, you will need to specify the `--split-by` option and instruct Sqoop to divide the import work among tasks based on values in the `order_id` field. An alternative is to use the `-m 1` option to force Sqoop to import all the data with a single task, but this would significantly reduce performance.

```
$ sqoop import \
 --connect jdbc:mysql://localhost/analyst_dualcore \
 --username training --password training \
 --fields-terminated-by '\t' \
 --warehouse-dir /analyst/dualcore \
 --table order_details \
 --split-by order_id
```

---

32. Finally, verify the above using the command:

```
$ hdfs dfs -ls /analyst/dualcore
```

You should have several subdirectories. Check that you have five named after the five MySQL tables that you imported. If any are missing, go back to the appropriate step and try again. (Don't just use the up arrow and re-run the command—if you

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

made a mistake in typing, you need to find and fix the error.) If you like, you can also list the contents of those directories to see that data has been transferred.

This is the end of the exercise.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

Hands-On Exercise: Running Queries from Shells, Scripts, and Hue

Files and Data Used in This Exercise

Exercise directory	`$ADIR/exercises/queries`
Hive/Impala tables	`customers, products`

In this exercise, you will practice using the Hue query editor and the Impala and Beeline shells to execute simple queries. These exercises use the tables that have been populated with data you imported to HDFS using Sqoop in an earlier exercise.

IMPORTANT: This exercise builds on previous ones. If you were unable to complete any previous exercise or think you may have made a mistake, run the following command to prepare for this exercise before continuing:

```
$ $ADIR/scripts/catchup.sh
```

Exploring Tables Using Hue

One way to run Hive and Impala queries is through your web browser using Hue's Query Editors. This is especially convenient if you use more than one computer—or if you use a device (such as a tablet) that isn't capable of running the Impala or Beeline shells itself—because it does not require any software other than a web browser.

1. Start the Firefox web browser if it isn't running, then click on the Hue bookmark in the Firefox bookmark toolbar (or type `http://localhost.localdomain:8889/hue` into the address bar and press the `Enter` key).

---

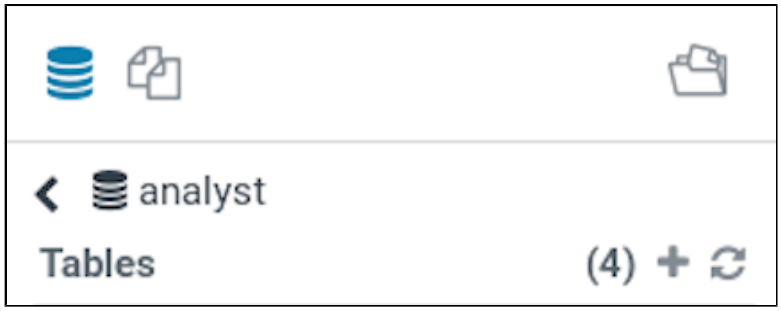
2. After a few seconds, you should see Hue's home screen. If you don't currently have an active session, you will first be prompted to log in. Enter `training` in both the username and password fields, and then click the **Sign In** button.

---

3. Click the **Editor** icon (`</>`) in the sidebar. Note that there are query editors for both Hive and Impala (as well as other tools). The interface is very similar for both Hive and Impala. For these exercises, you will use the Impala query editor, so choose **Impala**.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

4. The left panel shows available databases. Navigate to the `analyst` database in Hive or Impala, if it's not already selected. You might need to click the back arrow.



---

5. Below the selected database is a list of the tables in that database. Click the `customers` table to view the columns in the table. It may take a few seconds to display the columns.

---

6. Hover over the information **i** that appears next to the table name in the `customers` row. Notice the sample data from the table. This is a quick way in Hue to get some idea what the data looks like. (If no sample data appears, click the refresh icon in the top right corner.)

---

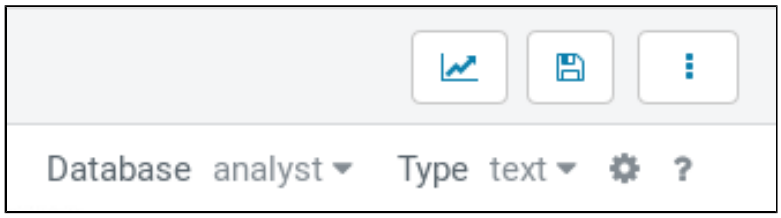
7. Click the **X** to close the information box, or just move your cursor off the table row and wait a moment.

---

8. Check the sample data for the other three tables, `order_details`, `orders`, and `products`. Click the refresh icon when necessary.

---

9. In the query editor panel, find the active database. This (not the left panel) specifies which database you are using with your queries. Make sure the `analyst` database is selected as the active database in the query editor panel. If necessary, select it yourself:



© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

Running a Query Using Hue

Dualcore ran a contest in which customers posted videos of interesting ways to use their new tablets. A $5,000 prize will be awarded to the customer whose video received the highest rating.

However, the registration data was lost due to an RDBMS crash, and the only information we have is from the videos. The winning customer introduced herself only as "Bridget from Kansas City" in her video.

You will need to run a query that identifies the winner's record in our customer database so that we can send her the $5,000 prize.

10. All you know about the winner is that her name is Bridget and she lives in Kansas City. In the Impala Query Editor, enter a query in the text area to find the winning customer. Run the following command search for Kansas City customers with names such as Bridget, Bridgette, or Bridgitte.

```
SELECT * FROM customers
  WHERE fname LIKE 'Bridg%' AND city = 'Kansas City';
```

---

11. After entering the query, click the **Execute** button (▶) to the left of the text area.

While the query is executing, the **Query History** tab displays the status of the query. When the query is complete, the **Results** tab opens, displaying the results of the query.

Question: Which customer did your query identify as the winner of the $5,000 prize?

Running a Query from the Impala Shell

Do the following to run a top-N query to identify the three most expensive products that Dualcore currently offers.

12. Start a terminal window if you don't currently have one running.

---

13. On the Linux command line in the terminal window, start the Impala shell:

```
$ impala-shell
```

Impala displays the URL of the Impala server in the shell command prompt, for example:

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

```
[localhost.localdomain:21000]>
```

---

14. At the prompt, switch to the `analyst` database by entering

```
> USE analyst;
```

Remember that SQL commands in the shell must be terminated by a semicolon (`;`), unlike in the Hue query editor.

---

15. Next, review the schema of the `products` table by entering

```
> DESCRIBE products;
```

---

16. Show a sample of 10 records from the products table:

```
> SELECT * FROM products LIMIT 10;
```

---

17. Write and execute a query that displays the three most expensive products. (**Hint:** Use `ORDER BY`.)

---

18. When you are done, exit the Impala shell:

```
> quit;
```

Running a Script in the Impala Shell

The rules for the contest described earlier require that the winner bought the advertised tablet from Dualcore between May 1, 2013 and May 31, 2013. Before we can authorize our accounting department to pay the $5,000 prize, you must ensure that Bridget is eligible. Since this query involves joining data from several tables, and we have not yet covered joins, you've been provided with a script in the exercise directory.

19. Change to the directory for this hands-on exercise:

```
$ cd $ADIR/exercises/queries
```

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

20. Review the code for the query:

```
$ cat verify_tablet_order.sql
```

---

21. Execute the script using the shell's `-f` option:

```
$ impala-shell -f verify_tablet_order.sql
```

Question: Did Bridget order the advertised tablet in May?

Running a Query Using Beeline

22. At the command line in a terminal window, start Beeline, using username **training** and password **training** when prompted. You might find it helpful to include `--silent=true` to suppress informational output, which is mostly unnecessary. Another option you might find helpful is `--showDbInPrompt=true`, which puts the name of the active database in the prompt so you can easily verify which is active.

```
$ beeline
 --silent=true --showDbInPrompt=true
```

For the exercise environment, you can use the `-u`, `-n`, and `-p` options if you wish, but they are not needed.

Beeline displays the URL of the Hive server in the shell command prompt, such as:

```
0: jdbc:hive2://localhost:10000 (default) >
```

If you include `--showDbInPrompt=true`, the active database will appear at the end in parenthesis. In the example above, the active database is `default`.

> **Warning Messages**
>
> When you start Beeline in the exercise environment, you will see several lines starting with `WARNING:` and `SLF4J:`. These can safely be ignored.

23. Switch to the `analyst` database.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

24. Write and execute a query to find all the Gigabux brand products in the `products` table whose price is less than 1000 (less than $10).

Hint: Use `WHERE price < 1000 AND ...`.

---

25. Exit the Beeline shell:

```
> !exit
```

This is the end of the exercise.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

Hands-On Exercise: Using Functions

Files and Data Used in This Exercise	
Exercise directory	`$ADIR/exercises/functions`
Impala/Hive Tables	`orders, products`

In this exercise, you will write queries using functions to analyze data in tables.

IMPORTANT: This exercise builds on previous ones. If you were unable to complete any previous exercise or think you may have made a mistake, run the following command to prepare for this exercise before continuing.

```
$ $ADIR/scripts/catchup.sh
```

Several analysis questions are described below, and you will need to run queries to answer them. Unless a particular tool (Hive or Impala) is specified, you can use whichever tool you prefer, using whichever method you like best, including shell, script, or the Hue Query Editor.

There are several ways you could write these queries. An example solution for each is provided in the `$ADIR/exercises/functions/solution/` directory.

Note that some of the questions—especially the bonus questions—are subject to interpretation.

Exploring the Dualcore Data

1. To begin, familiarize yourself with the `orders` table. You can look at the schema and get a sample of a few rows to see what the data looks like.

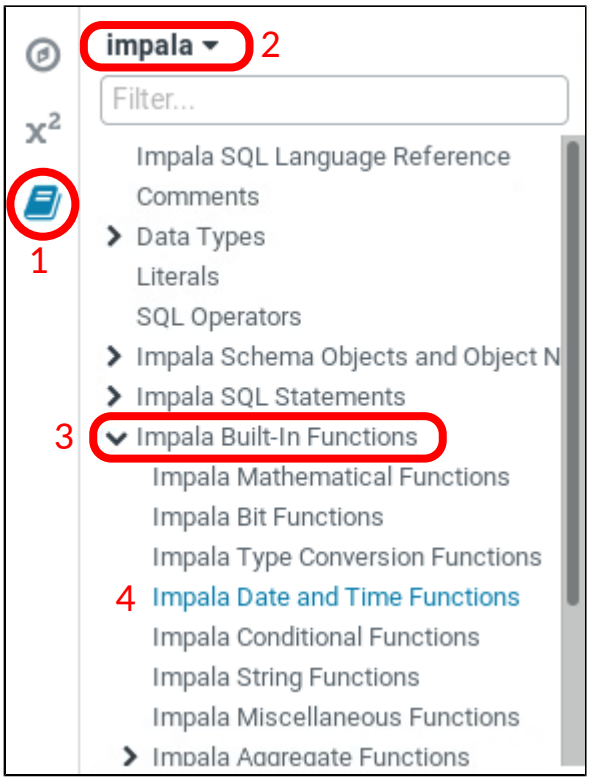
   You'll also work with the `products` table. If you need to familiarize yourself with that table as well, do so now.

Busy Times

The `order_date` column in the `orders` table is a `timestamp` column that provides the time of the order, down to the second. You can use this to determine which are the busy times of day.

2. Use the Language Reference for Impala embedded in Hue (see below) to find a function for extracting the hour from a `timestamp` type.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.



---

3. Test the function by finding the distinct values for hours in the `order_date` column. (You should have only `0` to `23`.)

---

4. Which are the three most active hours, and the three least active hours, for Dualcore orders?

Finding Profit

In the `products` table, the `price` column is what the customer pays for an item. The `cost` column gives the wholesale cost that Dualcore pays for the item. The *profit* on an item is the difference between what the customer pays and what Dualcore pays.

5. Find the five items that provide the largest profit. (They should all be servers.)

---

6. What items provide no profit, or actually cost Dualcore more than they charge for it? Use a filter to return only the rows for which this is the case. (Your query should return three rows.)

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

7. If either of the `price` or the `cost` column has a `NULL` value for a row, then that row would not be included in the previous queries. Do the following to check how complete this data is.

   - First check either of the columns for `NULL` values using a null operator. Does the column have any `NULL` values?

   - Now write a single query that will check both columns for `NULL` values. Are there any?

Averages

Answer the following about averages for different brands.

8. Write a query to find the average cost, average price, and average profit of all the items Dualcore carries. (The query shows an average profit of 3004.278276481149, or around $30.04.)

---

9. Modify the previous query to find the same averages for each brand *and* provide a better formatted result by rounding the averages to the nearest penny (that is, the nearest integer). (Your query should return 47 rows.)

---

10. Modify the query to filter the results, so you only get those brands with an average price greater than $1000 (`100000`). Write down the largest average price for the next problem. (Your query should return only two rows.)

---

11. Find the items whose price is larger than the largest average price that you found for the previous problem. (Your query should return 20 rows.)

Bonus Exercise: Price-to-Profit Ratio

If you have successfully finished the earlier steps and still have time, you can continue with this optional bonus exercise. At any time, if you find something you find interesting or curious, please feel free to investigate it on your own! If you need help with these queries, there are solutions in the `$ADIR/exercises/functions/bonus/solution`.

1. How much does the price-to-profit ratio vary among items? If you get a strange result, check your query and think about what might give that result.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

2. How much does it change within a brand?

---

3. What ten items have the largest price-to-profit ratio? (That is, they are relatively expensive for the amount of money they bring in to the company.)

---

4. Compare price-to-profit ratios for items similar to those. Purely from a price-to-profit perspective, would you recommend Dualcore stop carrying any particular items from a particular brand?

---

5. What other things might Dualcore need to consider before agreeing to stop carrying those items?

This is the end of the exercise.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

Hands-On Exercise: Data Management

Files and Data Used in This Exercise	
Exercise directory	`$ADIR/exercises/data_mgmt`
HDFS data directory	`/analyst/dualcore/employees`
Local data files	`$ADIR/data/ratings_2012.txt` `$ADIR/data/ratings_2013.txt`
Hive/Impala tables	`customers`

In this exercise, you will practice using several common techniques for creating and populating tables.

IMPORTANT: This exercise builds on previous ones. If you were unable to complete any previous exercise or think you may have made a mistake, run the following command to prepare for this exercise before continuing:

```
$ $ADIR/scripts/catchup.sh
```

Reviewing Existing Tables using Hue's Table Browser

1. In Firefox, visit the Hue home page, and then click the database icon in the sidebar to open the Table Browser.

---

2. Make sure the `analyst` database is selected.

---

3. Click the link for the `customers` table in the main panel to display the table overview and review the list of columns.

---

4. Click the **Sample** tab to view the first hundred rows of data.

Creating and Loading a Table using Hue's Table Browser

Hue's Table Browser can help you create a table and load it with data. Do the following to create and then load a table with product ratings data.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

5. Before creating the table, review the files containing the product ratings data. The files are in `/home/training/training_materials/analyst/data`. You can use the `head` command in a terminal window to see the first few lines:

```
$ head $ADIR/data/ratings_2012.txt
$ head $ADIR/data/ratings_2013.txt
```

---

6. Return to the Table Browser in Hue. If the sample data is still shown, click on **analyst** in the breadcrumbs to return to the table list. Click on the **+ New** button in the upper right to start the table definition wizard.

---

7. First, specify a data source that Hue will use to help create the table.

   a. Verify that **File** is selected for **Type**. If necessary, click the down arrow to choose **File**.

   b. Upload the file from your local machine to HDFS:

      i. Click the dots to the right of the **Path** field. This opens a pop-up window labeled **Choose a file**, with a list of directories and possibly files in HDFS.

      ii. Click **Upload a file** in the lower right corner.

      iii. Navigate the file system to find and select the `ratings_2012.txt` file. You can double-click on the file name to select it and upload to HDFS.

   c. After one or two seconds, the file will appear in the list of your pop-up window. Click on the file name to choose it and close the pop-up window. Hue will infer details about the file, and it will display a preview of the five fields that it finds.

   d. Click the **Next** button.

---

8. The next step is to set up the table specifications.

   a. Under **DESTINATION**, click in the **Name** field and name the new table `analyst.ratings`. This names the new table `ratings` and puts it in the `analyst` database.

   b. Under **PROPERTIES**, choose the file format. File formats will be covered later in the course. For now, **Format** should show **Text**. Correct it if needed.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

c. Click **Extras** to see the options provided there. This allows you to change the storage location for the data files, set a table to be transactional, add a description for your table, and set delimiters.

   i. Be sure the **Store in Default location** box is checked.

   ii. Uncheck the **Transactional table** box. (You cannot alter the **Insert only** box in this case, so just leave it as is.)

   iii. Be sure the **Import data** box is checked, so the `ratings_2012.txt` file will be loaded when you create the table.

   iv. *Optional:* You can include a description in the **Description** field, such as "Ratings data for products" if you like. This is not necessary.

   v. Verify the **Custom char delimiters** box is checked. This data is tab delimited, so verify that the **Field** is set for **^Tab( )** and change it if necessary. For this simple table, only the field terminator is relevant; the other delimiters are used for complex data and are not needed at this time.

d. Scroll down to the **Fields** section. Edit the fields so they have the names and types described below. Be sure to add the fields in the order given here.

Field Name	Field Type
`posted`	`timestamp`
`cust_id`	`int`
`prod_id`	`int`
`rating`	`tinyint`
`message`	`string`

e. When you have defined the columns, click **Submit**. This will start a task to define the table in the metastore and create the warehouse directory in HDFS to store the data.

f. When the task is complete, a **Task History** pop-up window will appear. Check that no error message is given, then click the **X** to close the pop-up window.

---

9. Verify that the table is in the `analyst` database. (With any of these steps, you might need to click the **Refresh** button or refresh the browser window.)

   a. Click **analyst** in the breadcrumbs and verify the table is now in the database list.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

b. Click on the table name to get the table overview and details.

c. Click **Sample** and review the results. Verify that the columns are correct and the data seems to be properly delimited.

---

10. Try querying the data in the table. In Hue, click the **Editors** button in the side bar and choose **Impala**.

Check that the current database in the query editor window is set to `analyst`. Set it if needed.

Try counting the number of ratings:

```
SELECT COUNT(*) FROM ratings;
```

The total number of records should be 464.

---

11. You can also load data to an existing table. One way to do this is in Hue; another is to use the `LOAD DATA INPATH` command. Try doing it using Hue.

a. First, copy the `ratings_2013.txt` data file to the `/analyst/dualcore` directory in HDFS. You may use either the Hue File Browser, or the `hdfs` command in the terminal window:

```
$ hdfs dfs -put $ADIR/data/ratings_2013.txt /analyst/dualcore/
```

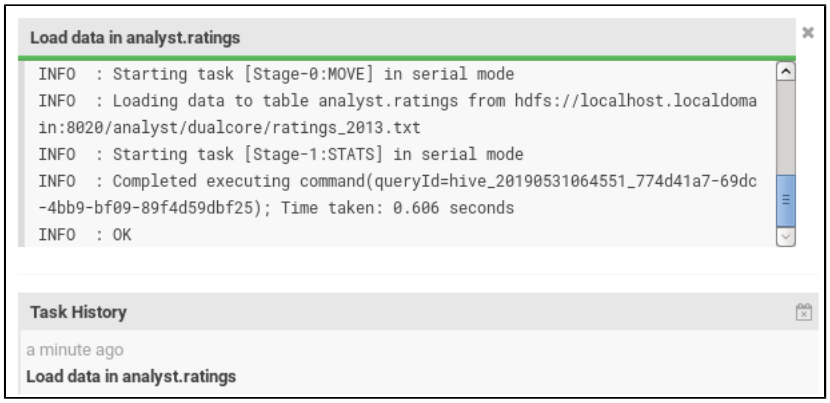
b. Return to the Hue Table Browser, and click the link for the new `ratings` table in the main panel.

c. Then click the **Import** button in the upper right corner.

d. In the **Import Data** dialog, enter or browse to the HDFS location of the 2013 product ratings data file: `/analyst/dualcore/ratings_2013.txt`. Be sure that the **Overwrite existing data** box is *not* checked, and then click **Submit**.

e. The **Task History** pop-up window should appear again. Check that the data loaded without error. It should look similar to this:

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.



Click the **X** to close the **Task History** window.

---

12. The `LOAD DATA INPATH` command and the Hue data import tool *move* the file to the table's directory. Using the Hue File Browser or `hdfs dfs` commands, verify that the file is no longer present in the original directory:

```
$ hdfs dfs -ls /analyst/dualcore/
```

---

13. *Optional:* Verify that the 2013 data is shown alongside the 2012 data in the table's warehouse directory.

---

14. With the additional data, there are now 21,997 records. Try counting the records in the ratings table again.

```
SELECT COUNT(*) FROM ratings;
```

Creating an External Table Using `CREATE TABLE`

The data for an `employees` table already exists in HDFS. In this section, you'll set up an external table so you can query this data.

In the last section, you practiced creating a table using the Hue Table Browser; this time, use an Impala SQL statement.

15. Using either the Impala shell or the Impala Query Editor in Hue, write and execute a `CREATE TABLE` statement to create an *external* table named `employees` for the records in HDFS at `/analyst/dualcore/employees`. Recall that the data is stored in *tab-delimited text* files.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

Field Name	Field Type
emp_id	STRING
fname	STRING
lname	STRING
address	STRING
city	STRING
state	STRING
zipcode	STRING
job_title	STRING
email	STRING
active	STRING
salary	INT

---

16. You can verify the details of the table you have created by issuing a `SHOW CREATE TABLE` statement:

```
SHOW CREATE TABLE employees;
```

Note how informative the result is: It includes many lower-level details about your table, including the directory location of the table data and the table properties.

---

17. Run the following query to verify that your table works properly.

```
SELECT job_title, COUNT(*) AS num
FROM employees
GROUP BY job_title
ORDER BY num DESC
LIMIT 3;
```

It should show that Sales Associate, Cashier, and Assistant Manager are the three most common job titles at Dualcore.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

Bonus Exercise: Creating, Loading, and Altering

If you have successfully finished the main exercise and still have time, feel free to continue with this bonus exercise. You can compare your work against the files found in the `bonus/solution/` subdirectory.

In this exercise, you will create a `suppliers` table, import the data using Sqoop, and then alter the table. You may complete these exercises using either the Impala shell or the Impala query editor in Hue.

1. Create a managed table named `suppliers` in the `analyst` database, with the following specifications.

   File type: Text
   Field delimiter: Tab (\t)
   Columns:

Field Name	Type
`supp_id`	`int`
`company`	`string`
`contact`	`string`
`address`	`string`
`city`	`string`
`state`	`string`
`zipcode`	`string`
`phone`	`string`

---

2. Use Sqoop to import the data for the `suppliers` table from the `suppliers` table in MySQL, with fields terminated by the tab character. Hints:
   - You need to use `--warehouse-dir` to specify the location to place the data. If you don't, Sqoop will put the data in the `/user/training/` directory in HDFS.
   - If you need help writing the command, examine the command in the `$ADIR/exercises/data_mgmt/bonus/solution/` directory.

---

3. It is always a good idea to verify that data has been added as intended. Execute a query to count the number of suppliers in Texas. The query should show that nine records match.

---

4. Use `ALTER TABLE` to rename the `company` column to `name`.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

5. Use the `DESCRIBE` command on the `suppliers` table to verify the change.

6. Use `ALTER TABLE` to rename the entire table to `vendors`.

7. Verify that your changes are properly applied by running a query on the table using the new names, for example:

```
SELECT supp_id, name FROM vendors LIMIT 10;
```

This is the end of the exercise.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

Hands-On Exercise: Data Storage and Performance

Files and Data Used in This Exercise

Exercise directory	`$ADIR/exercises/data_storage`
HDFS data directories	`/analyst/dualcore/ad_data1`, `/analyst/dualcore/ad_data2`
Local data directory	`$ADIR/data/latlon`

In this exercise, you will create a table for ad click data, partitioned by the network on which the ad was displayed.

Dualcore uses online advertisements to attract new customers to our e-commerce site. Each of the two ad networks we use provides data about the ads that have been placed. This includes the site where the ad was placed, the date when it was placed, what keywords triggered its display, whether the user clicked the ad, and the per-click cost.

Creating and Loading a Static Partitioned Table

The following HDFS directories contain processed data about the ad networks:

Ad Network 1: `/analyst/dualcore/ad_data1`
Ad Network 2: `/analyst/dualcore/ad_data2`

The ad networks are not identified by a field in the data; instead, the networks are distinguished by keeping the data from the different networks in separate files. Now, however, we would like to be able to query the ad data in the same table without losing the ability to distinguish between the two networks. We can do this by creating a partition for each network.

1. *Optional:* View the first few lines in the data files for both networks.

---

2. Define a *partitioned* table in the `analyst` database called `ads`, with the following characteristics:

   Type: `EXTERNAL`
   Partition column: `network` (type `TINYINT`)
   Field Terminator: `\t` (tab)
   File Format: Text file
   Location: `/analyst/dualcore/ads`
   Columns:

Field Name	Field Type
`campaign_id`	`STRING`

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

Field Name	Field Type
display_date	STRING
display_time	STRING
keyword	STRING
display_site	STRING
placement	STRING
was_clicked	TINYINT
cpc	INT

3. Alter the `ads` table to add two partitions, one for network 1 and one for network 2.

4. Load the data in `/analyst/dualcore/ad_data1` into the Network 1 partition.

5. Load the data in `/analyst/dualcore/ad_data2` into the Network 2 partition.

6. Verify that the data for both ad networks were correctly loaded by counting the records for each:

```
SELECT COUNT(*) FROM ads WHERE network=1;
SELECT COUNT(*) FROM ads WHERE network=2;
```

Note: If you are using Hue and put multiple commands in the Query Editor at the same time, Hue runs them one at a time. After it runs a command, Hue will report the results, then wait for another click of the **Execute** button before continuing.

Network 1 should have **438,389** records and Network 2 should have **350,563** records.

Bonus Exercise: Accessing Data in Parquet File Format

If you have successfully finished the earlier steps and still have time, feel free to continue with this optional bonus exercise.

For this exercise, you have been provided data in Parquet format that is output from another tool—in this example, the output from an ETL process that maps US zip codes to latitude and longitude. You need to be able to query this data in Impala.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

1. The ETL data output directory is provided as `$ADIR/data/latlon`. Copy the data directory to the `/analyst/dualcore` directory in HDFS. You can use the Hue File Browser, or the following `hdfs` command:

```
$ hdfs dfs -put $ADIR/data/latlon /analyst/dualcore/
```

---

2. Using Impala, create an external Parquet-based table in the `analyst` database pointing to the `latlon` data directory.

   - Hint: The actual data in the data directory is in a MapReduce output file called `part-m-000000_0.parquet`. Use the `LIKE PARQUET` command to use the existing column definitions in the data file.

---

3. Review the table in Impala to confirm it was correctly created with columns `zip`, `latitude` and `longitude`.

---

4. Perform a query or preview the data in the Impala Query Editor to confirm that the data in the data file is being accessed correctly.

This is the end of the exercise.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

Hands-On Exercise: Working with Multiple Datasets

Exercise directory	`$ADIR/exercises/multiple_datasets`

In this exercise, you will use joins to analyze sales and product data.

IMPORTANT: This exercise builds on previous ones. If you were unable to complete any previous exercise or think you may have made a mistake, run the following command to prepare for this exercise before continuing:

```
$ $ADIR/scripts/catchup.sh
```

Several analysis questions are described below, and you will need to run queries to answer them. You can use whichever tool you prefer—Hive or Impala—using whichever method you like best, including shell, script, or the Hue Query Editor.

You will need to use more than one table in the `analyst` database in the exercise environment.

There are several ways you could interpret some of these questions, and different ways you can write the queries. One possible solution for each is in the `solution` directory.

Calculating the Top N Products

- Which top three products have been sold more than any other products? (Hint: Remember that if you use a `GROUP BY` clause, you must group by all fields listed in the `SELECT` clause that are not part of an aggregate function.)

Calculating the Order Total

- Which ten orders had the highest total dollar amounts?

Calculating Revenue and Profit

- Run a query to show Dualcore's revenue (total price of products sold) and profit (price minus cost) by date.
  - Hint: The `order_date` column in the `orders` table is of type `TIMESTAMP`. Use the function `to_date` to get just the date portion of the value.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

Bonus Exercise: Daily Profits by Month

If you have successfully finished the earlier steps and still have time, feel free to continue with this optional bonus exercise.

Note: If you need help, you can find solutions in the `$ADIR/exercises/multiple_datasets/bonus/solution` directory.

- Run a query to show each day's profit. Limit the results to 20 rows.

- Run a query to give only the daily profits for May, 2013, in chronological order.

  Hint: You'll need to use the `HAVING` clause or the `WHERE` clause. There are several ways to match the date. For example, the `to_date` function will return the date element of a timestamp as a string, and then you can use `BETWEEN … AND` for the comparison operator, or use <= and >=. You could also extract and match the month and year separately and use the = operator.

- Run a query to show the profits by month, in order chronologically.

  Hint: You want to order by year first, and then month.

- Find the four most profitable months and the four least profitable months.

This is the end of the exercise.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

Hands-On Exercise: Analytic Functions

Exercise directory	`$ADIR/exercises/analytic_functions`

IMPORTANT: This exercise builds on previous ones. If you were unable to complete any previous exercise or think you may have made a mistake, run the following command to prepare for this exercise before continuing:

```
$ $ADIR/scripts/catchup.sh
```

Background Information

Dualcore uses online advertisements to attract new customers to our e-commerce site. Each of the two ad networks we use provides data about the ads that have been placed. This includes the site where the ad was placed, the date when it was placed, what keywords triggered its display, whether the user clicked the ad, and the per-click cost.

1. Review the `ads` table to familiarize yourself with its contents.

Time-Based Windows

Analyzing data over time is a common use of windowing. For each of the following questions, write a query to answer it, using windowing and analytic functions.

2. How many ads were displayed per site per day?

   Hint: You don't need to use analytic functions or windowing for this one.

---

3. For each day, how did each site rank in number of displays?

   Hint: Use your previous query as a subquery.

   Hint: To make it easy to see if you have done this correctly, first sort by day, and then by rank within each day.

---

4. For each day, how did each site's daily count compare to the previous day?

   - The results should display the site's daily count and the previous day's count in the same row.
   - Put the sites together in alphanumerical order, and then within each site, put the daily counts in order.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

5. What is the per-site average count for a seven-day period?

   Hint: Use a sliding window (with `ROWS BETWEEN …`)

Bonus Exercise: Finding Medians

Recall that the *median* of a dataset is the value which half the data is above, and half is below. For a dataset with an odd number of points, the median is the point in the exact middle. In this set of seven points, the median is the highlighted 4; there are three points below it and three points above it:
1, 2, 2, 4, 4, 8, 12

For a dataset with an even number of points, the median is typically the average of the two middle points. In this set of eight points, the highlighted 2 and 4 are the two middle points, so the median is (2 + 4)/2, or 3:
1, 2, 2, 2, 4, 4, 8, 12

In Apache Hive, you can calculate percentiles for integers; `percentile(column, 0.5)` will produce the median. However, neither Hive nor Apache Impala have a function that calculates a precise median for non-integers. For large datasets, it's a complex operation, so approximate medians are typically accurate enough.

Both Hive and Impala have functions to *approximate* the median for non-integers. For Hive, you can use `percentile_approx(column, 0.5)`; for Impala, you can use `APPX_MEDIAN(column)`.

You can use workarounds with the analytic functions to find your own approximation of the median, but the resulting queries are a bit complex.

1. Use the `CUME_DIST` function in a subquery to find the median price of all the products in the `products` table. Your query should output only one row, with only one column.

   Hint: This can sometimes provide the exact median, but it's not guaranteed. The issue is finding a point where the cumulative distribution is *exactly* 0.5. If you get zero rows returned, you might try relaxing comparison expressions.

---

2. Modify your query to find the median price for each brand in the `products` table. Your query should return one row for each brand. You might want to review the analytic functions.

This is the end of the exercise.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

Hands-On Exercise: Complex Data

Files and Data Used in This Exercise	
Exercise directory	`$ADIR/exercises/complex_data`
Local data file	`$ADIR/data/loyalty_data.txt`

In this exercise, you will load and query data with complex columns related to a customer loyalty program.

IMPORTANT: This exercise builds on previous ones. If you were unable to complete any previous exercise or think you may have made a mistake, run the following command to prepare for this exercise before continuing:

```
$ $ADIR/scripts/catchup.sh
```

Creating, Loading, and Querying Complex Data with Hive

Dualcore recently started a loyalty program to reward our best customers. A colleague has already provided us with a sample of the data that contains information about customers who have signed up for the program, including their phone numbers (as a `MAP`), a list of past order IDs (as an `ARRAY`), and a `STRUCT` that summarizes the minimum, maximum, average, and total value of past orders (in cents). You will use Hive to create the table, populate it with the provided data, and then run a few queries to practice referencing these types of fields. Finally, you will create a Parquet table containing the same data, to allow you to use Impala to query the data.

You may use either the Beeline shell or Hue's Hive Query Editor for these exercises.

1. In the `analyst` database, create a text file-based table with the following characteristics:

   Name: `loyalty_program`
   Type: `EXTERNAL`
   Field Terminator: | (pipe)
   Collection Item Terminator: , (comma)
   Map Key Terminator: : (colon)
   Location: `/analyst/dualcore/loyalty_program`
   Columns:

Field Name	Field Type
`cust_id`	`INT`
`fname`	`STRING`
`lname`	`STRING`

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

Field Name	Field Type
email	STRING
level	STRING
phone	MAP<STRING,STRING>
order_ids	ARRAY<INT>
order_value	STRUCT<min:INT, max:INT, avg:INT, total:INT>

2. Using either a terminal window or the Pluma editor, examine the data in `$ADIR/data/loyalty_data.txt` to see how it corresponds to the fields in the table.

3. Load the data file by placing it into the HDFS warehouse directory for the new table. You can use either the Hue File Browser, or the `hdfs` command:

```
$ hdfs dfs -put $ADIR/data/loyalty_data.txt \
  /analyst/dualcore/loyalty_program
```

4. Using either the Beeline shell or Hue's Hive Query Editor, run a query to select the `HOME` phone number for customer ID 1200866. (Hint: `MAP` keys are case-sensitive.) You should see 408-555-4914 as the result.

5. Select the third element from the `order_ids` field (of `ARRAY` type) for customer ID 1200866. (Hint: elements are indexed from zero.) The query should return 5278505.

6. Select the `total` attribute from the `order_value` field (of `STRUCT` type) for customer ID 1200866. The query should return 401874.

7. Create an *external*, Parquet-based table named `loyalty_program_parquet` with the same characteristics as the `loyalty_programtable`, and use Hive to copy all the data from the `loyalty_program` table into this new `loyalty_program_parquet` table. (Hint: You can use `CREATE TABLE AS SELECT` (CTAS) to do this with a single HiveQL statement.)

Querying Complex Data in a Parquet Table with Impala

Now that you have used Hive to create, load, and query data with complex columns, continue by using Impala to query complex data in a Parquet-based table. Remember

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

that the Impala SQL syntax for queries involving `ARRAY` and `MAP` types differs from the HiveQL syntax.

You may use either the Impala shell or Hue's Impala Query Editor to complete these exercises.

8. Run a query to return the distinct key values in the `phone` column. (Hint: You can query a `MAP` column in Impala as if it were a separate table with columns `key` and `value`.)

---

9. These distinct key values represent types of phone numbers. Now run a query to count how many of each type of phone number there are in the `phone` column.

---

10. Run a query to count the total number of order IDs listed in the `order_ids` column. (Hint: You can query an `ARRAY` column in Impala as if it were a separate table with columns `item` and `pos`.)

---

11. Select the first element in the `order_ids` field (of `ARRAY` type) for each customer. (Hint: The `pos` column represents the position of each element in an `ARRAY`, indexed from zero.)

---

12. Run a query to count how many customers have 30 or more orders.

Bonus Exercise 1: Hive Table-Generating Functions

If you have successfully finished the earlier steps and still have time, feel free to continue with this optional bonus exercise.

1. Run a Hive query to return the order IDs for customer ID 1200866, one per row. (Hint: Use the `explode` function.)

---

2. Run a Hive query to return the customer IDs and order IDs for all customers who have an average order total of greater than $900 (90,000 cents). The query should return one order ID per row. (Hint: Use `LATERAL VIEW`.)

Bonus Exercise 2: Impala Complex Column Join Notation

If you have successfully finished the earlier steps and still have time, feel free to continue with this optional bonus exercise.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

1. Run an Impala query to return the same result set as the previous Hive query: the customer IDs and order IDs for all customers who have an average order total of greater than $900. (Hint: Use join notation, and remember to query the Parquet-based table.)

---

2. Now run an Impala query that returns the customer IDs, first names, last names, phone numbers, and phone number types for these customers who have an average order of greater than $900.

This is the end of the exercise.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

Hands-On Exercise: Analyzing Text with Hive

Files and Data Used in This Exercise	
Exercise directory	`$ADIR/exercises/text_analysis`
Local data file	`$ADIR/data/access.log`
Hive/Impala tables	`ratings`

In this exercise, you will use use Hive to process and analyze web server log data and to analyze product ratings.

IMPORTANT: This exercise builds on previous ones. If you were unable to complete any previous exercise or think you may have made a mistake, run the following command to prepare for this exercise before continuing:

```
$ $ADIR/scripts/catchup.sh
```

Creating and Populating the Web Logs Table

Many interesting analyses can be done on the data from the usage of a website. The first step is to load the semi-structured data in the web log files into a Hive table. Typical log file formats are not delimited, so you will need to use the Regex SerDe and specify a pattern Hive can use to parse lines into individual fields you can then query.

You may use either Beeline or Hue's Hive Query Editor to complete these exercises.

1. Examine `create_web_logs.hql` to get an idea of how the `CREATE TABLE` command uses a Regex SerDe to parse lines in the log file (an example log line is shown in the comment at the top of the file). When you have examined the script, run it to create the table. You can paste the code into the Hive Query Editor, or use Beeline in a terminal window:

```
$ beeline -u jdbc:hive2://localhost:10000 \
 -n training -p training \
 -f $ADIR/exercises/text_analysis/create_web_logs.hql
```

---

2. Populate the table by adding the log file to the table's directory in HDFS:

```
$ hdfs dfs -put $ADIR/data/access.log \
/analyst/dualcore/web_logs/
```

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

3. Verify that the data is loaded correctly by running this query to show the top three items users searched for on our website:

```
SELECT term, COUNT(term) AS num
FROM
  (SELECT lower(regexp_extract(request,
      '/search\\?phrase=(\\S+)', 1)) AS term
   FROM web_logs
   WHERE request REGEXP '/search\\?phrase=') term
GROUP BY term
ORDER BY num DESC
LIMIT 3;
```

You should see that this query returns `tablet` (`303`), `ram` (`153`), and `wifi` (`148`).

Note: The `REGEXP` operator, which is available in some SQL dialects, is similar to `LIKE`, but uses regular expressions for more powerful pattern matching. The `REGEXP` operator is synonymous with the `RLIKE` operator.

Bonus Exercise 1: Analyzing Numeric Product Ratings

If you have successfully finished the earlier steps and still have time, feel free to continue with this optional bonus exercise.

Customer ratings and feedback are great sources of information for both customers and retailers like Dualcore. However, customer comments are typically free-form text and must be handled differently than structured data. Fortunately, Hive provides extensive support for text processing.

Before delving into text processing, you'll begin by analyzing the numeric ratings customers have assigned to various products. In the next bonus exercise, you will use these results in doing text analysis.

1. Review the `ratings` table structure using the Hive Query Editor or using the `DESCRIBE` command in the Beeline shell.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

2. We want to find the product that customers like most, but we must guard against being misled by products that have few ratings assigned. Run the following query to find the product with the highest average among all those with at least 50 ratings:

```
SELECT prod_id, format_number(avg_rating, 2) AS avg_rating
FROM (SELECT prod_id, AVG(rating) AS avg_rating,
        COUNT(*) AS num
      FROM ratings
      GROUP BY prod_id) rated
WHERE num >= 50
ORDER BY avg_rating DESC
LIMIT 1;
```

---

3. Rewrite, and then execute, the query above to find the product with the *lowest* average among products with at least 50 ratings. You should see that the result is product ID 1274673 with an average rating of 1.10.

Bonus Exercise 2: Analyzing Rating Comments

We observed in the previous bonus exercise that customers are very dissatisfied with one of the products we sell (the product with ID 1274673). Although numeric ratings helped us identify *which* product that is, they cannot tell us *why* customers don't like the product. We could simply read through all the comments associated with that product to learn this information, but that approach doesn't scale. Next, you will use Hive's text processing support to analyze the comments.

1. The following query normalizes all comments on that product to lowercase, breaks them into individual words using the `sentences` function, and passes those to the `ngrams` function to find the five most common bigrams (two-word combinations). Run the query:

```
SELECT explode(ngrams(sentences(lower(message)), 2, 5))
        AS bigrams
FROM ratings
WHERE prod_id = 1274673;
```

---

2. Most of these words are too common to provide much insight, though the word *expensive* does stand out in the list. Modify the previous query to find the five most common *trigrams* (three-word combinations), and then run that query in Hive.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

3. Among the patterns you see in the result is the phrase *ten times more.* This might be related to the complaints that the product is too expensive. Now that you've identified a specific phrase, look at a few comments that contain it by running this query:

```
SELECT message
FROM ratings
WHERE prod_id = 1274673
     AND lower(message) LIKE '%ten times more%'
LIMIT 3;
```

You should see comments that say, "Why does the red one cost ten times more than the others?" and "Red is ten times more expensive than the others!"

---

4. We can infer that customers are complaining about the price of the red-color version of this item. Write and execute a query that will find all distinct comments containing the word *red* that are associated with product ID 1274673.

---

5. The previous step should have displayed three comments:

   - "Red is ten times more expensive than the others!"
   - "What is so special about red?"
   - "Why does the red one cost ten times more than the others?"

   The first and third comment imply that this product is overpriced relative to similar products. Write and run a query that will display the record for product ID 1274673 in the `products` table.

---

6. Your query should have shown that the product was `16GB USB Flash Drive (Red)` from the Orion brand. Next, run this query to identify similar products:

```
SELECT *
FROM products
WHERE name LIKE '%16 GB USB Flash Drive%'
    AND brand='Orion';
```

The query results show that we have three products that are almost identical, differing only in color. However, the product with the negative reviews (the red one) is priced about ten times higher than the others, just as some of the comments said.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

The costs for these differently colored products are the same, but the price of the red one is 42999 cents ($429.99) whereas the prices of the other two are both 4299 cents ($42.99). It appears that using text processing on the product reviews may have helped us to uncover a pricing error.

This is the end of the exercise.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

Hands-On Exercise: Hive Optimization

Files and Data Used in This Exercise	
Exercise directory	`$ADIR/exercises/hive_optimization`
Hive/Impala table	`products`

In this exercise, you will practice techniques to improve Hive query performance.

IMPORTANT: This exercise builds on previous ones. If you were unable to complete any previous exercise or think you may have made a mistake, run the following command to prepare for this exercise before continuing:

```
$ $ADIR/scripts/catchup.sh
```

Dualcore sells 1,114 different products. The Marketing Director needs your help to identify which products are most popular and which are least popular, so that advertising campaigns can be planned for both sets of products.

IMPORTANT: Use a single session of Beeline or Hue's Hive Query Editor to complete all the steps of this exercise. Except for this one session, close all other sessions of Beeline and close all other web browser windows containing Hue's Hive Query Editor.

Examining an Execution Plan

In this section, you will look at execution plans for a query that finds the ten most popular products.

1. The following Hive query returns the top 10 products in terms of quantities sold. Paste this HiveQL into Beeline or Hue's Hive Query Editor and execute the query:

```
SELECT name, brand, o.prod_id, num
FROM
   (SELECT prod_id, COUNT(order_id) AS num
    FROM order_details
    GROUP BY prod_id
    ORDER BY num DESC
    LIMIT 10) o
  JOIN products p ON o.prod_id = p.prod_id;
```

---

2. Make a note of the amount of time the query takes to complete. (You will compare it with another run later.) Note that if you invoke beeline without the "--silent" option, Hive will follow the records returned with a line reporting the number of rows in

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

the result and the time to run the query. Or, you can just time the query informally yourself.

---

3. Examine Hive's execution plan for the query.

a. Display the plan by prefixing your query with EXPLAIN or by using the **Explain** button (📖) in Hue.

b. Copy the results and paste into a text document. You can use that document to make comments, highlight sections, and compare the plan to another version of the plan later.

c. Note what the stage dependencies are, and compare that to the order in which the stages are explained.

d. Do your best to identify how each stage corresponds to the query. (The list of items below to look for will help.) Don't worry if you aren't able to understand all the parts of the execution plan; this takes practice and experience. For any execution tasks you are unsure about, you might compare it to the parts of the query that you could not match.

Things to look for:

- `TableScan` (What table is scanned?)
- `SELECT Operator` (What columns or expressions are selected?)
- `Group By Operator` (What are the aggregations and what are the keys?)
- `Limit` (How many rows?)
- `Fetch Operator` (What appears to be happening here?)
- `Join Operator` (What kind of join operator is being used?)

Using Sorted ORC Files

In this section, you will create a sorted ORC files version of the same tables and see what effects, if any, there are on query performance.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

4. Run the following to create new tables, storing the data as ORC files, and then insert the data using sorted files.

```
CREATE TEMPORARY TABLE order_details_orc
   (order_id INT, prod_id INT)
  STORED AS ORC;

CREATE TEMPORARY TABLE products_orc
   (prod_id INT, brand STRING,
    name STRING, price INT,
    cost INT, shipping_wt INT)
  STORED AS ORC;

INSERT INTO order_details_orc
   SELECT * FROM order_details SORT BY prod_id;

INSERT INTO products_orc
   SELECT * FROM products SORT BY prod_id;
```

---

5. Modify the query to find the most popular items, using the new tables instead.

---

6. Run the modified query and compare the run time to the performance of the original query.

Using Bloom Filters

In this section, you will create new versions of the tables with Bloom filters turned on and see what effects, if any, there are on query performance.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

7. Examine this query to see how it finds all products that are included in the same order as the most popular product, counts them, and presents the 20 most popular.

```
SELECT name, brand, p.prod_id, num
FROM products p
  JOIN
    (SELECT o.prod_id AS prod_id, COUNT(order_id) AS num
     FROM order_details o
     WHERE o.order_id IN
       (SELECT order_id
        FROM order_details
        WHERE prod_id = 1274348)
     GROUP BY o.prod_id) x
    ON p.prod_id = x.prod_id
ORDER BY num DESC
LIMIT 20;
```

---

8. Run the query and note the time it takes to run it.

---

9. Use the following to create a new `order_details_bloom` table using ORC files with a Bloom filter on the `prod_id` column, and give it the same data as the existing `order_details` table.

```
CREATE TEMPORARY TABLE order_details_bloom
      (order_id INT, prod_id INT)
    STORED AS ORC
    TBLPROPERTIES ('orc.bloom.filter.columns'='prod_id');

INSERT INTO order_details_bloom
    SELECT * FROM order_details;
```

---

10. Modify the query to use the new `order_details_bloom` table. (Note that there are *two* places where you need to make a change.) Run the query and compare the time it takes to what it took with the original table.

This is the end of the exercise.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

Hands-On Exercise: Impala Optimization

Files and Data Used in This Exercise	
Exercise directory	`$ADIR/exercises/impala_optimization`
Hive/Impala tables	`products, ads`

In this exercise, you will explore query execution plans for various types of queries.

IMPORTANT: This exercise builds on previous ones. If you were unable to complete any previous exercise or think you may have made a mistake, run the following command to prepare for this exercise before continuing:

```
$ $ADIR/scripts/catchup.sh
```

Reviewing Query Execution Plans

1. Review the execution plan for the following query. You may use either the Impala Query Editor in Hue or the Impala shell command line tool.

```
SELECT * FROM products;
```

---

2. Note that the query explanation includes a warning that table and column stats are not available for the `products` table. Compute the stats by executing the following command:

```
COMPUTE STATS products;
```

---

3. Now view the query plan again, this time without the warning.

---

4. The previous query was very simple, and included only a single table in the `FROM` clause. Try reviewing the query plan of a more complex query. The following query

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

returns the top three products sold. Before using `EXPLAIN`, compute stats on the tables to be queried.

```
SELECT brand, name, COUNT(p.prod_id) AS sold
FROM products p
JOIN order_details d
    ON (p.prod_id = d.prod_id)
GROUP BY brand, name, p.prod_id
ORDER BY sold DESC
LIMIT 3;
```

Questions: How many stages are there in this query? What are the estimated per-host memory requirements for this query? What is the total size of all partitions to be scanned?

---

5. The tables in the queries above each have only a single partition. Try reviewing query plans for a partitioned table. The `ads` table is partitioned on the `network` column. First, compute stats for this table:

```
COMPUTE STATS ads;
```

---

6. Now compare the query plans for the following two queries. The first calculates the total cost of *clicked ads* for each ad campaign; the second does the same, but for *all* ads on *one* of ad networks.

```
SELECT campaign_id, SUM(cpc)
FROM ads
WHERE was_clicked=1
GROUP BY campaign_id
ORDER BY campaign_id;
```

```
SELECT campaign_id, SUM(cpc)
FROM ads
WHERE network=2
GROUP BY campaign_id
ORDER BY campaign_id;
```

Questions: What is the estimate per-host memory requirements for the two queries? What explains the difference?

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

Bonus Exercise: Reviewing the Query Summary

If you have successfully finished the earlier steps and still have time, feel free to continue with this optional bonus exercise.

1. Start Impala Shell in a terminal window.

---

2. Try executing one of the queries you examined above; for example:

```
SELECT brand, name, COUNT(p.prod_id) AS sold
FROM products p
JOIN order_details d
  ON (p.prod_id = d.prod_id)
GROUP BY brand, name, p.prod_id
ORDER BY sold DESC
LIMIT 3;
```

---

3. After the query completes, execute the SUMMARY command:

```
SUMMARY;
```

---

4. **Questions:** Which stage took the longest average time to complete? Which took the most memory?

---

5. Now run the same query in Hue, using the Impala editor.

---

6. Between the query editor and the results is an info window, which will have a blue link. Click the link to review the information provided there.

   a. First click the **Plan** tab. This gives a Directed Acyclic Graph (DAG) representation of the execution plan.

   b. Then click the **Summary** tab. Compare the results to the results of the SUMMARY command you ran in Impala Shell.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

This is the end of the exercise.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

Hands-On Exercise: Data Transformation with Hive

Files and Data Used in This Exercise	
Exercise directory	`$ADIR/exercises/transform`
Hive/Impala table	`web_logs`

In this exercise, you will use a Hive transform script and a UDF to estimate shipping costs on abandoned orders.

IMPORTANT: This exercise builds on previous ones. If you were unable to complete any previous exercise or think you may have made a mistake, run the following command to prepare for this exercise before continuing:

```
$ $ADIR/scripts/catchup.sh
```

Analyzing Customer Checkouts

As on many websites, Dualcore's customers add products to their shopping carts and then follow a checkout process to complete their purchase. We want to figure out if customers who start the checkout process are completing it. Since each part of the four-step checkout process can be identified by its URL in the logs, we can use a regular expression to identify them:

Step	Request URL	Description
1	`/cart/checkout/step1-viewcart`	View list of items added to cart
2	`/cart/checkout/step2-shippingcost`	Notify customer of shipping cost
3	`/cart/checkout/step3-payment`	Collect payment information
4	`/cart/checkout/step4-receipt`	Show receipt for completed order

Note: Because the `web_logs` table uses a Regex SerDe, which is a feature not supported by Impala, this section of the exercise must be completed using Hive. You may use either the Beeline shell or the Hive Query Editor in Hue.

1. Run the following query in Hive to show the number of requests for each step of the checkout process:

```
SELECT COUNT(*), request
FROM web_logs
WHERE request REGEXP '/cart/checkout/step\\d.+'
GROUP BY request;
```


© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

The results of this query highlight a problem: about one out of every three customers abandons their cart after the second step. This might mean millions of dollars in lost revenue, so let's see if we can determine the cause.

---

2. The log file's `cookie` field stores a value that uniquely identifies each user session. Since not all sessions involve checkouts at all, create a new table containing the session ID and number of checkout steps completed for just those sessions that do:

```
CREATE EXTERNAL TABLE checkout_sessions AS
 SELECT cookie, ip_address, COUNT(request) AS steps_completed
 FROM web_logs
 WHERE request REGEXP '/cart/checkout/step\\d.+'
 GROUP BY cookie, ip_address;
```

---

3. Run this query to show the number of people who completed only one checkout step (view cart), only two checkout steps (see shipping cost), or all four checkout steps:

```
SELECT steps_completed, COUNT(cookie) AS num
FROM checkout_sessions
GROUP BY steps_completed;
```

You should see from the differences in these numbers that many customers abandon their order after the second step, which is when they first learn how much it will cost to ship their order.

---

4. *Optional:* Because the new `checkout_sessions` table does not use a SerDe, it can be queried in Impala. Try running the same query as in the previous step in Impala. What happens?

Using TRANSFORM for IP Geolocation

Based on what you've just seen, it seems likely that customers abandon their carts due to high shipping costs. The shipping cost is based on the customer's location and the weight of the items they've ordered. Although this information isn't in the database (since the order wasn't completed), you can gather enough data from the logs to estimate them.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

You don't have the customer's address, but you can use a process known as *IP geolocation* to map the computer's IP address in the log file to an approximate physical location. Since this isn't a built-in capability of Hive, you'll use a provided Python script to `TRANSFORM` the `ip_address` field from the `checkout_sessions` table to a ZIP code, as part of a HiveQL statement that creates a new table called `cart_zipcodes`.

> **Regarding TRANSFORM and UDF Examples in This Exercise**
>
> During this exercise, you will use a Python script for IP geolocation and a UDF to calculate shipping costs. Both are implemented merely as a simulation—compatible with the fictitious data we use in class and intended to work even when Internet access is unavailable. The focus of these exercises is on how to *use* external scripts and UDFs, rather than how the code for the examples works internally.

5. In a terminal window, use the following command to change the directory to the exercise directory:

```
$ cd $ADIR/exercises/transform/
```

---

6. Examine the `create_cart_zipcodes.hql` script in the exercise directory, and observe the following:

a. It creates a new table called `cart_zipcodes` based on a `SELECT` statement.

b. The `SELECT` statement transforms the `cookie`, `ip_address`, and `steps_completed` fields from the `checkout_sessions` table using a Python script (which you'll look at in the next step).

c. The new table contains the ZIP code instead of an IP address, plus the other two fields from the original table.

---

7. Examine the `ipgeolocator.py` script and observe the following:

a. Records are read from Hive on standard input.

b. The script splits them into individual fields using a tab delimiter.

c. The `ip_addr` field is converted to `zipcode`, but the `cookie` and `steps_completed` fields are passed through unmodified.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

d. The three fields in each output record are delimited with tabs and printed to standard output.

---

8. Copy the Python file to HDFS so that the Hive Server can access it. You may use the Hue File Browser or the `hdfs dfs` command:

```
$ hdfs dfs -put ipgeolocator.py /analyst/dualcore/
```

---

9. Run the script to create the `cart_zipcodes` table. You can either paste the code into the Hive Query Editor, or use Beeline in a terminal window:

```
$ beeline -u jdbc:hive2://localhost:10000 \
  --silent=true -f create_cart_zipcodes.hql
```

Extracting a List of Products Added to Each Cart

As described earlier, estimating the shipping cost also requires a list of items in the customer's cart. You can identify products added to the cart since the request URL looks like this (only the product ID changes from one record to the next):

```
/cart/additem?productid=1234567
```

10. Write and execute a HiveQL statement to create and load an external table called `cart_items` with two fields: `cookie` and `prod_id` (of type `int`) based on data selected the `web_logs` table. Keep the following in mind when writing your statement:

    a. The `prod_id` field should contain only the seven-digit product ID. (Hint: Use the `regexp_extract` function. Note that this will make a `string` column, so you need to do something else to make it `int`.)

    b. Use a `WHERE` clause with `REGEXP` using the same regular expression as above, so that you only include records where customers are adding items to the cart.

    c. If you need a hint on how to write the statement, look at the `create_cart_items.hql` file in the exercise's `solution` directory.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

11. Verify the contents of the new table by running this query:

```
SELECT COUNT(DISTINCT cookie) FROM cart_items
WHERE prod_id=1273905;
```

If this doesn't return 47, compare your statement to the `create_cart_items.hql` file, make the necessary corrections, and then re-run your statement (after dropping the `cart_items` table).

Creating Tables to Join Web Logs with Product Data

You now have tables representing the ZIP codes and products associated with checkout sessions, but you'll need to join these with the `products` table to get the weight of the products before you can estimate shipping costs. In order to do some more analysis later, you'll also include total selling price and total wholesale cost in addition to the total shipping weight for all items in the cart.

12. Run the following HiveQL to create a table called `cart_orders` with this information:

```
CREATE EXTERNAL TABLE cart_orders AS
   SELECT z.cookie, steps_completed, zipcode,
          SUM(shipping_wt) AS total_weight,
          SUM(price) AS total_price,
          SUM(cost) AS total_cost
   FROM cart_zipcodes z
     JOIN cart_items i
       ON (z.cookie = i.cookie)
     JOIN products p
       ON (i.prod_id = p.prod_id)
   GROUP BY z.cookie, zipcode, steps_completed;
```

Creating a Table Using a UDF to Estimate Shipping Costs

You finally have all the information you need to estimate the shipping cost for each abandoned order. One of the developers on our team has already written, compiled, and packaged a Hive UDF that will calculate the shipping cost given a ZIP code and the total weight of all items in the order.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

13. Before you can use a UDF, you must make it available to Hive. First, copy the file to HDFS so that the Hive Server can access it. You may use the Hue File Browser or the `hdfs dfs` command:

```
$ hdfs dfs -put geolocation_udf.jar /analyst/dualcore/
```

---

14. Next, register the function with Hive and provide the name of the UDF class as well as the alias you want to use for the function. Run the Hive command below to associate the UDF with the alias `calc_shipping_cost`:

```
CREATE FUNCTION calc_shipping_cost
   AS 'com.cloudera.hive.udf.UDFCalcShippingCost'
   USING JAR 'hdfs:/analyst/dualcore/geolocation_udf.jar';
```

---

15. Now create a new external table called `cart_shipping` that will contain the session ID, number of steps completed, total retail price, total wholesale cost, and the estimated shipping cost for each order based on data from the `cart_orders` table:

```
CREATE EXTERNAL TABLE cart_shipping AS
   SELECT cookie, steps_completed, total_price, total_cost,
    calc_shipping_cost(zipcode, total_weight) AS shipping_cost
   FROM cart_orders;
```

---

16. Finally, verify your table by running the following query to check a record:

```
SELECT * FROM cart_shipping WHERE cookie='100002920697';
```

This should show that session as having two completed steps, a total retail price of $263.77, a total wholesale cost of $236.98, and a shipping cost of $9.09.

Note: The `total_price`, `total_cost`, and `shipping_cost` columns in the `cart_shipping` table contain the number of cents as integers. Be sure to divide results containing monetary amounts by 100 to get dollars and cents.

This is the end of the exercise.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

Optional Hands-On Exercise: Analyzing Abandoned Carts

Files and Data Used in This Exercise

Exercise directory	`$ADIR/exercises/abandoned_carts`
Hive/Impala tables	`cart_shipping, products, order_details, orders`

In this exercise, you will analyze data about abandoned orders to determine if a free shipping promotion would be profitable.

Dualcore is concerned about a large number of visitors to their online store who abandon their carts after the second step, when they see the shipping costs.

IMPORTANT: This exercise builds on previous ones. If you were unable to complete any previous exercise or think you may have made a mistake, run the following command to prepare for this exercise before continuing:

```
$ $ADIR/scripts/catchup.sh
```

You will need to use Impala for at least some of this exercise because it uses a feature not available in Hive. If you prefer to use Hive, you may do so until noted otherwise. You may use the shell, scripts, or the Hue Query Editor.

Analyzing Lost Revenue

1. First, calculate how much revenue the abandoned carts represent. There are four steps in the checkout process, so only records in the `cart_shipping` table with a `steps_completed` value of four represent a completed purchase:

   ```
   SELECT SUM(total_price) AS lost_revenue
   FROM cart_shipping
   WHERE steps_completed < 4;
   ```

   You should see that abandoned carts mean that Dualcore is potentially losing out on more than $2 million in revenue. Clearly it's worth the effort to do further analysis.

   Note: The `total_price`, `total_cost`, and `shipping_cost` columns in the `cart_shipping` table contain the number of cents as integers. Be sure to divide results containing monetary amounts by 100 to get dollars and cents.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

2. The number returned by the previous query is revenue, but what counts is profit. Gross profit for an item is the price minus the cost. Write and execute a query similar to the one above, but which reports the total lost profit from abandoned carts. If you need a hint on how to write this query, you can check the `solution/abandoned_checkout_profit.sql` file.

   - After running your query, you should see that Dualcore is potentially losing $111,058.90 in profit due to customers not completing the checkout process.

---

3. How does this compare to the amount of profit Dualcore receives from customers who do complete the checkout process? Modify your previous query to consider only those records where `steps_completed = 4`, and then re-execute it. Check `solution/completed_checkout_profit.sql` if you need a hint.

   - The result should show that we earn a total of $177,932.93 on completed orders, so abandoned carts represent a substantial proportion of additional profits.

---

4. The previous two queries returned the *total* profit for abandoned and completed orders, but these aren't directly comparable because there were different numbers of each. It might be the case that one is much more profitable than the other on a per-order basis. Write and execute a query that will calculate the *average* profit based on the number of steps completed during the checkout process. If you need help writing this query, check the `solution/avg_profit_by_step.sql` file.

   - You should observe that carts abandoned after step 2 represent an even higher average profit per order than completed orders.

Calculating Cost and Profit for a Free Shipping Offer

You have observed that most carts—and on average, the most *profitable* carts—are abandoned at the point where we display the shipping cost to the customer. You will now run some queries to determine whether offering free shipping, on at least some orders, would actually bring in more profit, assuming this offer prompted more customers to finish the checkout process.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

5. Run the following query to compare the average shipping cost for orders abandoned after the second step versus completed orders:

```
SELECT steps_completed, AVG(shipping_cost) AS avg_ship_cost
FROM cart_shipping
WHERE steps_completed IN (2,4)
GROUP BY steps_completed;
```

- You will see that the average shipping cost for abandoned orders was almost 10% higher than for completed purchases. Offering free shipping, at least for some orders, might actually bring in more money than passing on the cost to customers and risking abandoned orders.

---

6. Run the following query to determine the average profit per order over the entire month for the data you are analyzing in the log file. This will help you to determine whether we could absorb the cost of offering free shipping:

```
SELECT AVG(order_profit) AS avg_profit_per_order
FROM
   (SELECT SUM(price - cost) AS order_profit
    FROM products p
      JOIN order_details d ON d.prod_id = p.prod_id
      JOIN orders o ON d.order_id = o.order_id
    WHERE YEAR(order_date) = 2013 AND
          MONTH(order_date) = 5
    GROUP BY o.order_id) s;
```

- You should see that the average profit for all orders during May was about $16.80. It seems clear that Dualcore could afford to offer free shipping. However, it may be more advantageous to only do so on orders over a certain amount.

Note: From this point on, the queries rely on an alias in the `SELECT` list that is used in the `GROUP BY` clause. Hive will not allow this, so the given queries will not work in Hive. If you have been using Hive, **please switch now to Impala.** (If you want an added challenge, you can continue to use Hive, but you'll need to modify the provided queries, either repeating the aliased expression in the `GROUP BY` clause or using a subquery so the alias can be used. Also note, however, that the modified queries will take longer in Hive than in Impala.)

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

7. Run the following query, which is a slightly revised version of the previous one, to compare the average profit for orders greater than or less than $10:

```
SELECT case
         when order_price <= 1000 then 'up to $10'
         when order_price > 1000 then 'over $10'
         else 'unknown'
       end AS price_tier,
       AVG(order_profit) AS avg_profit_per_order
FROM
  (SELECT SUM(price) as order_price,
          SUM(price - cost) AS order_profit
   FROM products p
     JOIN order_details d ON d.prod_id = p.prod_id
     JOIN orders o ON d.order_id = o.order_id
   WHERE YEAR(order_date) = 2013 AND
         MONTH(order_date) = 5
   GROUP BY o.order_id) s
GROUP BY price_tier;
```

- You should see that our average profit on orders of $10 or under was only $0.86, but for orders more than $10, it was $17.05. We probably would not want to offer free shipping on those lower orders, because the profit is already small.

---

8. Modify the previous query's `case` statement to separate orders above $10 into those up to $50, $100, $500, $1000, and above $1000 (`solution/avg_profit_per_order_tiered.sql` contains the solution, in case you need a hint). Execute your new statement.

- You should see that each tier jumps a fair amount, and then there is a huge spike from an average of $40 for those between $500 and $1000 to an average of $241 on average for those above $1000).

---

9. The `cart_shipping` table contains each order's total price, cost, shipping weight, and shipping cost. How much does average shipping cost compare to the average profits (price minus cost) for orders in each of these tiers? Write and run a query to calculate average shipping cost and average profit for each tier (`solution/avg_shipping_cost_tiers.sql` contains the solution, in case you need a hint).

- You should see that the average shipping cost is more than the average profit for orders up to $50. For orders up to $500, shipping is about half the profit, and

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

for up to $1000 it's more than a third of the profit. With orders over $1000, the shipping cost is $18.16, about 22% of the $81.43 profit.

---

10. Since there's no way to know in advance who will abandon their cart, Dualcore would have to absorb the cost on *all* orders above whichever price is decided as the cutoff for free shipping. Would the extra money from abandoned carts offset the added cost of free shipping for customers who would have completed their purchases anyway? Run the following query to see the total profit on completed purchases for each tier (combining the lower three):

```
SELECT case
         when total_price <= 10000 then 'up to $100'
         when total_price <= 50000 then 'over $100 up to $500'
         when total_price <= 100000 then 'over $500 up to $1000'
         when total_price > 100000 then 'over $1000'
         else 'unknown'
       end AS price_tier,
       SUM(total_price - total_cost) AS total_profit
FROM cart_shipping
WHERE steps_completed = 4
GROUP BY price_tier;
```

- After running this query, you should see that the total profit for completed orders is around $62K for the over $100 up to $500 tier, around $68K for the over $500 up to $1000 tier, and around $39K for orders above $1000.

---

11. Now, run the following query to find the potential profit for each tier, after subtracting shipping costs, if all customers completed the checkout process:

```
SELECT case
         when total_price <= 10000 then 'up to $100'
         when total_price <= 50000 then 'over $100 up to $500'
         when total_price <= 100000 then 'over $500 up to $1000'
         when total_price > 100000 then 'over $1000'
         else 'unknown'
       end AS price_tier,
       SUM(total_price - total_cost - shipping_cost)
         AS potential_profit
FROM cart_shipping
GROUP BY price_tier;
```


© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

As expected, Dualcore clearly would lose a lot of money by offering free shipping on orders under $100. The potential profit for orders between $100 and $500 is only around $52K, about $10K less than the profit from the completed orders that pay for shipping, so they would lose money by paying shipping for those orders as well. For orders between $500 and $1000, there would be a modest increase of $1000 (from $68K to $69K). The risk might not be worth it for that modest increase. But the result for orders over $1000 is $51K compared to the current profit of $39K from completed orders, so it appears that Dualcore could earn nearly $12,000 more by offering free shipping for all orders of at least $1000.

Congratulations! Your hard work analyzing a variety of data with Hadoop's tools has helped make Dualcore more profitable than ever.

This is the end of the exercise.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

Hands-On Exercise: Using Apache Kudu with Apache Impala

Files and Data Used in This Exercise

Exercise directory	`$ADIR/exercises/kudu`

Impala/Hive Tables	`names`
	`analyst_loudacre.accounts_hive`
	`analyst_loudacre.devices_hive`
	`analyst_loudacre.device_status_hive`

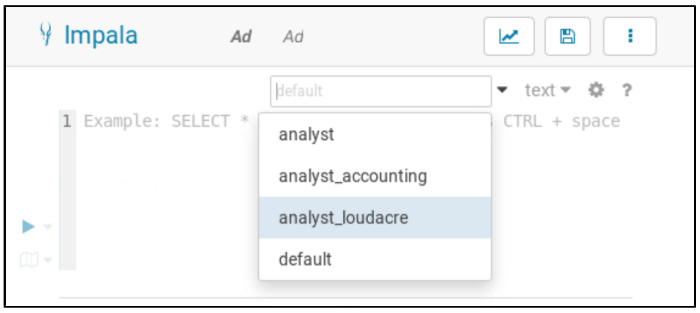
In this exercise, you will use Impala to create, load, and query Kudu tables. You may use Hue or Impala Shell for this exercise.

Create a Simple Example Table

1. Start Impala Shell or go to the Impala query editor in Hue.

---

2. Make the `analyst_loudacre` database your active database. In Hue, choose it from within the query editor window, as shown here.



Confirm that **analyst_loudacre** is now shown as the current database.

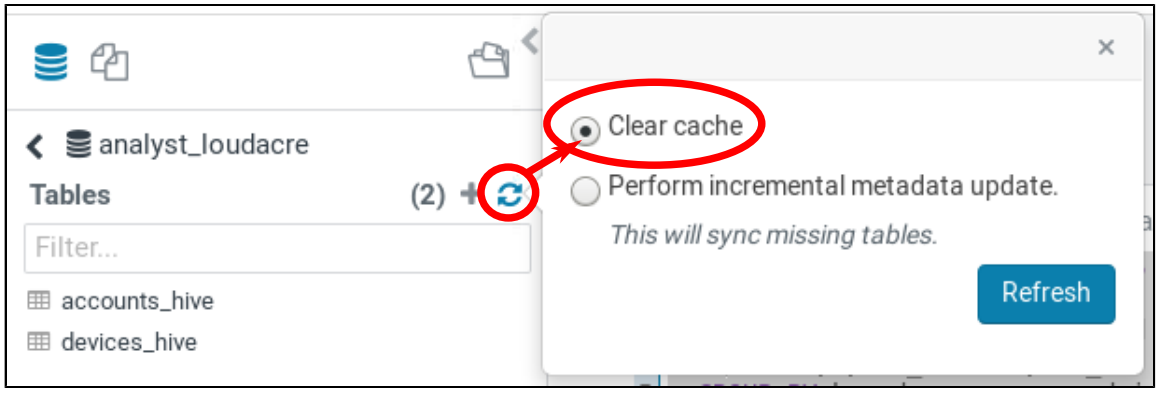
© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

3. Run the following SQL command to create a new table with two columns—`name` (string) and `age` (integer). The table should have a single primary key—`name`—and use hashing to distribute rows into two partitions.

```
CREATE TABLE names
  (
    name STRING,
    age INT,
    PRIMARY KEY(name)
  )
  PARTITION BY HASH PARTITIONS 2
  STORED AS KUDU;
```

---

4. Verify that the new `names` table is in the database. In Impala Shell, use `SHOW TABLES`; in Hue, it should appear in the the table browser on the left. You may need to navigate to the correct database, or click the **Refresh** button and select **Clear cache** to display the new table.



---

5. Run `DESCRIBE FORMATTED names;` and do the the following.
   - Confirm that the table's schema is correct.
   - Note that although you didn't use the `EXTERNAL` keyword, the table type is external.
   - Also note the existence of the `external.table.purge` property, set to `true`. Recall what this means for this external table.

When you created the Impala `names` table using `STORED AS KUDU` in the previous steps, Impala automatically created a corresponding Kudu table called `impala::analyst_loudacre.names`. You can view the Kudu table in the Kudu UI.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

6. In your browser, go to the Kudu master UI main page (http://localhost:8051).

---

7. Select the **Tables** item in the menu bar. Confirm that the `impala::analyst_loudacre.names` table is listed. (It should be the only one.)

---

8. Click the new table's **Table Id** link to see table details. Review the table's schema and partitioning strategy.

---

9. Click **toggle** under **Detail** to see more table details. How many tablets does the new table have? Is that the correct number?

Complete the following steps to add some data to the `names` table in Impala and then query the table.

10. Add a few rows of data into the `names` table using the `INSERT` SQL command. For example:

```
INSERT INTO names VALUES
  ('Grace Hopper',52),
  ('Ada Lovelace',28),
  ('Alan Turing',32);
```

---

11. Check the messages that the query produces. Confirm that there were no warnings or errors.

---

12. Run a query that returns a subset of the data in the `names` table. For example:

```
SELECT * FROM names WHERE age > 30;
```

Review the results and confirm that they are what you should expect.

---

13. Try inserting a new row with an existing key (name).

```
INSERT INTO names VALUES ('Grace Hopper',63);
```

Check the query messages for errors or warnings again.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

View the data in the table again. Does the new row appear?

---

14. Two rows cannot share the same key. Try "upserting" a row, which will insert rows with new keys and modify rows with existing keys.

```
UPSERT INTO names VALUES
  ('Grace Hopper',63),
  ('Katherine Johnson',99);
```

View the rows in the table again. Do you see what you expected?

---

15. Before you move on to the next section, drop the example table you created; it is not required for the rest of the exercises.

```
DROP TABLE names;
```

---

16. Confirm that the Impala `names` table was removed from the list of tables (in Hue, you might need to refresh the table browser). Then view the list of tables in the Kudu Master UI (`http://localhost:8051`) and confirm that the `impala::analyst_loudacre.names` Kudu table was dropped as well.

The Kudu table was dropped when the Impala table was dropped because the `external.table.purge` property was set to `true`.

Create a New Kudu Table with Data from an Existing Impala/Hive Table

In this section, you will create a new Kudu table containing data from an existing Impala/Hive table. You will start by reviewing the existing `accounts_hive` table.

17. Confirm that you are still using the `analyst_loudacre` database, and check the list of tables. Note that some Impala/Hive tables (based on data stored in HDFS) have been pre-created for this exercise.

---

18. Review the schema and some sample data in the `accounts_hive` table.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

19. Execute the following query to create a new `accounts_kudu` Impala table and the corresponding Kudu table using the schema and data of the existing `accounts_hive` Impala/Hive table.

```
CREATE TABLE accounts_kudu
    PRIMARY KEY (acct_num)
    PARTITION BY HASH PARTITIONS 2
    STORED AS KUDU
  AS SELECT * FROM accounts_hive;
```

Take note of the number of rows inserted in the new table.

---

20. Confirm that the `accounts_kudu` table was created correctly by viewing its schema and a sample of its data. Does it match the `accounts_hive` table it was created from?

Create a New Table with a Composite Key

As in the previous section, you will create a new Impala Kudu table based on an existing Impala table, this time using `devices_hive`. This table contains a list of the make, model, release date, and type (phone or tablet) of all the devices Loudacre supports.

21. Review the `devices_hive` schema and data.

In the previous section, you created a table using the account number as the key. The values in the `devnum` column are unique, making them a possible choice as a primary key for the new Kudu table. However, Loudacre performs most of the queries on this table by model name rather than device number, so a more efficient key is `model`. However, the values in the `model` column are not unique, therefore the table must use a composite primary key composed of the `make` and `model` columns.

22. Create another Impala Kudu table based on `devices_hive` as follows:

- Use table name `devices_composite`.
- Define a composite primary key using the `make` and `model` columns. (Use `PRIMARY KEY(make, model)`.)
- Use hash partitioning using `model` as the partition key column.

Note that the previous section used `*` in the `SELECT` statement in order to select all the columns in the order they were in the original table. However, Kudu requires that columns used as primary keys must be defined first in the table. So in this case,

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

you need to select all the columns explicitly using the correct order so that `make` and `model` are first.

When the command completes, view the schema and data in the new Kudu table to confirm that it was created correctly.

Create a New Table Using Range Partitioning

In this section, you will continue working with the data in the `analyst_loudacre.devices_hive` table. The Kudu table you created above used hash partitioning, which makes saving the data efficient, but date-based queries will need to scan every partition. To increase query performance, you will create a new Impala Kudu table that partitions by date ranges instead of hashing.

23. The `released` column in `devices_hive` contains a timestamp for when each device was first released. Partition the table by the devices' years of release. In order to do that, convert the timestamp to an integer representing the year using the SQL `YEAR` function. The year column must be part of the primary key so that you can use it as a partition key, but the year alone is not unique, so combine it with the device number to define a unique composite key.

Execute this command to create the table with partitions defined for the years 2016 and 2017, and a single partition for all years preceding 2016:

```
CREATE TABLE devices_range
    PRIMARY KEY (devnum,rel_year)
    PARTITION BY RANGE(rel_year) (
      PARTITION VALUES < 2016,
      PARTITION 2016 <= VALUES < 2017,
      PARTITION 2017 <= VALUES < 2018
    )
    STORED AS KUDU
  AS SELECT devnum,YEAR(released) as rel_year,
    released,make,model,dev_type
  FROM devices_hive;
```

---

24. Review the schema and partitioning of the new table in the Kudu UI (http://localhost:8051). Confirm that it has the expected number of partitions and tablets.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

25. With time- or date-based partitioning, you need to add a new partition to include data with a timestamp later than the existing defined partitions. Add a new partition for the next year's worth of data.

```
ALTER TABLE devices_range
ADD RANGE PARTITION 2018 <= VALUES < 2019;
```

Confirm using the Kudu UI (http://localhost:8051) that the additional partition was added to the table.

This is the end of the exercise.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

Appendix: Data Model Reference

Tables Imported from MySQL

The following depicts the structure of the MySQL tables imported into HDFS using Sqoop. The primary key column from the database, if any, is denoted by bold text:

`customers`: 201,375 records (imported to `/analyst/dualcore/customers`)

Index	Field	Description	Example
0	**`cust_id`**	Customer ID	`1846532`
1	`fname`	First name	`Sam`
2	`lname`	Last name	`Jones`
3	`address`	Address of residence	`456 Clue Road`
4	`city`	City	`Silicon Sands`
5	`state`	State	`CA`
6	`zipcode`	Postal code	`94306`

`employees`: 61,712 records (imported to `/analyst/dualcore/employees` and later used as an external table in Hive)

Index	Field	Description	Example
0	**`emp_id`**	Employee ID	`BR5331404`
1	`fname`	First name	`Betty`
2	`lname`	Last name	`Richardson`
3	`address`	Address of residence	`123 Shady Lane`
4	`city`	City	`Anytown`
5	`state`	State	`CA`
6	`zipcode`	Postal Code	`90210`
7	`job_title`	Employee's job title	`Vice President`
8	`email`	email address	`br5331404@example.com`
9	`active`	Is actively employed?	`Y`
10	`salary`	Annual pay (in dollars)	`136900`

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

orders: 1,662,951 records (imported to `/analyst/dualcore/orders`)

Index	Field	Description	Example
0	**order_id**	Order ID	3213254
1	cust_id	Customer ID	1846532
2	order_date	Date/time of order	2013-05-31 16:59:34

order_details: 3,333,244 records (imported to `/analyst/dualcore/order_details`)

Index	Field	Description	Example
0	order_id	Order ID	3213254
1	prod_id	Product ID	1754836

products: 1,114 records (imported to `/analyst/dualcore/products`)

Index	Field	Description	Example
0	**prod_id**	Product ID	1273641
1	brand	Brand name	Foocorp
2	name	Name of product	4-port USB Hub
3	price	Retail sales price, in cents	1999
4	cost	Wholesale cost, in cents	1463
5	shipping_wt	Shipping weight (in pounds)	1

suppliers: 66 records (imported to `/analyst/dualcore/suppliers`)

Index	Field	Description	Example
0	**supp_id**	Supplier ID	1000
1	company	Company name	ACME Inc.
2	contact	Full name of contact	Sally Jones
3	address	Address of office	123 Oak Street
4	city	City	New Athens
5	state	State	IL
6	zipcode	Postal code	62264
7	phone	Office phone number	(618) 555-5914

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

Hive/Impala Tables

The following is a record count for tables that are created or queried during the hands-on exercises. Use the `DESCRIBE` *`tablename`* command to see the table structure.

Table Name	Record Count
ads	788,952
cart_items	33,812
cart_orders	12,955
cart_shipping	12,955
cart_zipcodes	12,955
checkout_sessions	12,955
customers	201,375
employees	61,712
latlon	42968
loyalty_program	311
loyalty_program_parquet	311
order_details	3,333,244
orders	1,662,951
products	1,114
ratings	21,997
suppliers (renamed vendors)	66
web_logs	412,860

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

Other Data Added to HDFS

The following describes the structure of other important datasets added to HDFS.

Combined Ad Campaign Data: (788,952 records total), stored in two directories:

- `/analyst/dualcore/ad_data1` (438,389 records)
- `/analyst/dualcore/ad_data2` (350,563 records).

Index	Field	Description	Example
0	`campaign_id`	Uniquely identifies our ad	`A3`
1	`date`	Date of ad display	`05/23/2013`
2	`time`	Time of ad display	`15:39:26`
3	`keyword`	Keyword that triggered ad	`tablet`
4	`display_site`	Domain where ad shown	`news.example.com`
5	`placement`	Location of ad on web page	`INLINE`
6	`was_clicked`	Whether ad was clicked	`1`
7	`cpc`	Cost per click, in cents	`106`

`access.log`: 412,860 records (uploaded to `/analyst/dualcore/access.log`) This file is used to populate the `web_logs` table in Hive. Note that the RFC 931 and Username fields are seldom populated in log files for modern public websites and are ignored in our Regex SerDe.

Index	Field / Description	Example
0	IP address	`192.168.1.15`
1	RFC 931 (Ident)	`-`
2	Username	`-`
3	Date/Time	`[22/May/2013:15:01:46 -0800]`
4	Request	`"GET /foo?bar=1 HTTP/1.1"`
5	Status code	`200`
6	Bytes transferred	`762`
7	Referrer	`"http://dualcore.com/"`
8	User agent (browser)	`"Mozilla/4.0 [en] (WinNT; I)"`
9	Cookie (session ID)	`"SESSION=8763723145"`

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

Appendix: Regular Expression Reference

The following is a brief tutorial intended for the convenience of students who don't have experience using regular expressions or may need a refresher. A more complete reference can be found in the documentation for Java's `Pattern` class:

http://tiny.cloudera.com/regexpattern

Introduction to Regular Expressions

Regular expressions are used for pattern matching. There are two kinds of patterns in regular expressions: literals and metacharacters. Literal values are used to match precise patterns while metacharacters have special meaning; for example, a dot will match any single character. Here's the complete list of metacharacters, followed by explanations of those that are commonly used:

```
< ( [ { \ ^ - = $ ! | ] } ) ? * + . >
```

Literal characters are any characters not listed as a metacharacter. They're matched exactly, but if you want to match a metacharacter, you must escape it with a backslash. Since a backslash is itself a metacharacter, it must also be escaped with a backslash. For example, you would use the pattern `\\.` to match a literal dot.

Regular expressions support patterns much more flexible than simply using a dot to match any character. The following explains how to use *character classes* to restrict which characters are matched.

Character Classes

`[057]`	Matches any single digit that is either 0, 5, or 7
`[0-9]`	Matches any single digit between 0 and 9
`[3-6]`	Matches any single digit between 3 and 6
`[a-z]`	Matches any single lowercase letter
`[C-F]`	Matches any single uppercase letter between `C` and `F`

For example, the pattern `[C-F][3-6]` would match the string `D3` or `F5` but would fail to match `G3` or `C7`.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

There are also some built-in character classes that are shortcuts for common sets of characters.

Predefined Character Classes

`\\d`	Matches any single digit
`\\w`	Matches any word character (letters of any case, plus digits or underscore)
`\\s`	Matches any whitespace character (space, tab, newline, etc.)

For example, the pattern `\\d\\d\\d\\w` would match the string `314d` or `934X` but would fail to match `93X` or `Z871`.

Sometimes it's easier to choose what you don't want to match instead of what you do want to match. These three can be negated by using an uppercase letter instead.

Negated Predefined Character Classes

`\\D`	Matches any single non-digit character
`\\W`	Matches any non-word character
`\\S`	Matches any non-whitespace character

For example, the pattern `\\D\\D\\W` would match the string `ZX#` or `P @` but would fail to match `93X` or `36_`.

The metacharacters shown above match each exactly one character. You can specify them multiple times to match more than one character, but regular expressions support the use of quantifiers to eliminate this repetition.

Matching Quantifiers

`{5}`	Preceding character may occur exactly five times
`{0,6}`	Preceding character may occur between zero and six times
`?`	Preceding character is optional (may occur zero or one times)
`+`	Preceding character may occur one or more times
`*`	Preceding character may occur zero or more times

By default, quantifiers try to match as many characters as possible. If you used the pattern `ore.+a` on the string `Dualcore has a store in Florida`, you might be surprised to learn that it matches `ore has a store in Florida` rather than `ore ha` or `ore in Florida` as you might have expected. This is because matches are "greedy" by default. Adding a question mark makes the quantifier match as few characters as possible instead, so the pattern `ore.+?a` on this string would match `ore ha`.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

Finally, there are two special metacharacters that match zero characters. They are used to ensure that a string matches a pattern only when it occurs at the beginning or end of a string.

Boundary Matching Metacharacters

`^`	Matches only at the beginning of a string
`$`	Matches only at the ending of a string

Note: When used inside square brackets (which denote a character class), the `^` character is interpreted differently. In that context, it negates the match. Therefore, specifying the pattern `[^0-9]` is equivalent to using the predefined character class `\\D` described earlier.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

Appendix: Troubleshooting Tips

This appendix offers some tips to handle problems you may encounter while completing exercises. The issues addressed are:

- Services on the cluster are unavailable
- Cloudera Manager displays unhealthy status of services
- HDFS error running catchup script
- Kudu errors

Issue: Services on the cluster are unavailable

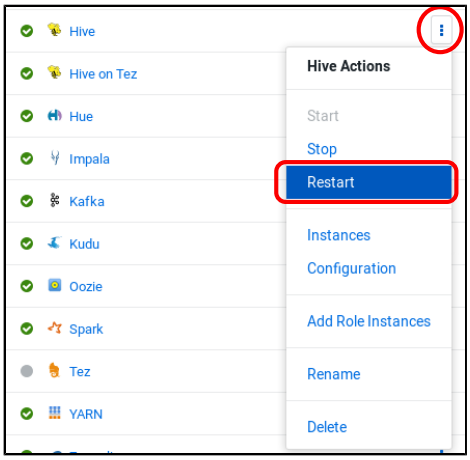
You may get errors in Hue or on the command line about being unable to connect to services such as Hue, YARN, HDFS, or Kudu, or other required services. In this case, the services may need to be restarted using Cloudera Manager.

1. Use the Cloudera Manager bookmark on your remote desktop browser to view the Cloudera Manager web UI, and log in using username **admin** with password **admin**.

---

2. If any of the cluster services you need for the exercises are shown with anything *other* than a green dot (such as a gray or red dot), restart the service by clicking on the dropdown menu next to the service name and selecting **Restart**.

   This screenshot shows an example in which the **HDFS** service is stopped, and how you would restart it.



© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

Issue: Cloudera Manager displays unhealthy status of services

If you just restarted the environment from a stopped state, it can take a few minutes for bad health status indicators to resolve themselves. However, if health issues persist, here are some ways to determine what might be causing the health issues to appear, and how to resolve them.

Click on **All Health Issues**, then click on **Organize by Health Test**.

- If you have **Process Status** issues where the Cloudera Manager agent is not responding (as indicated by **Hosts** with unhealthy status), try restarting the cluster services. From the desktop, select **Applications** > **Training** > **Start Cloudera Cluster** to restart your cluster services. Give the script time to complete.
- If you have any type of **Canary** issues, these typically clear up on their own, given time.
- If any other issues still exist after solving any process status issues:
  - In Cloudera Manager, note the name of one of the services reporting the issue (such as HDFS).
  - From the **Clusters** menu, choose that service (such as HDFS).
  - From the **Actions** menu, choose **Restart**.

Issue: HDFS error running catchup script

```
WARN hdfs.DFSClient: Caught exception
java.lang.InterruptedException
```

You may get this warning from the catchup script. This is just a warning, and you can disregard it.

Issue: Kudu errors

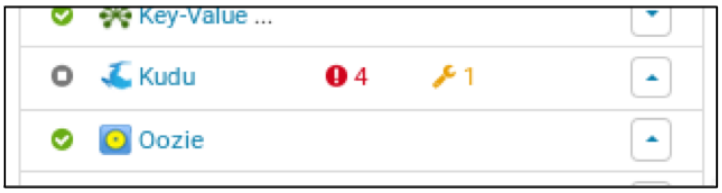
Your Impala command may hang, or you may get a number of different errors indicating that one or more Kudu servers is not responding, such as

- `Not enough live tablet servers to create a table`
- `NoLeaderFoundException`
- `Timed out: Could not connect to the cluster`
- `Connection refused`

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

- `Operation timed out`

Check if the Kudu service has an error condition using Cloudera Manager. In your web browser, use the **Cloudera Manager** bookmark, and log in as **admin** (password **admin**). The services in the cluster will display. Working services will display a green dot indicating the service is healthy. Non-working services will display a red or grey dot indicating that the service is not healthy.



If the **Kudu** service is unhealthy, follow the instructions in the Issue: Services on the cluster are unavailable section.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.

Appendix: Continuing Exercises After Class

After this class has finished, you may wish to continue working on the exercises or practice the skills you have learned.

The cloud-based exercise environment you were provided as part of the class will continue to be available after the class is over. You can continue to use the environment for up to ten hours within 30 days after the class is complete. If you require additional flexibility, please contact Cloudera.

© Copyright 2010–2020 Cloudera. All Rights Reserved.
Not to be reproduced or shared without prior written consent from Cloudera.